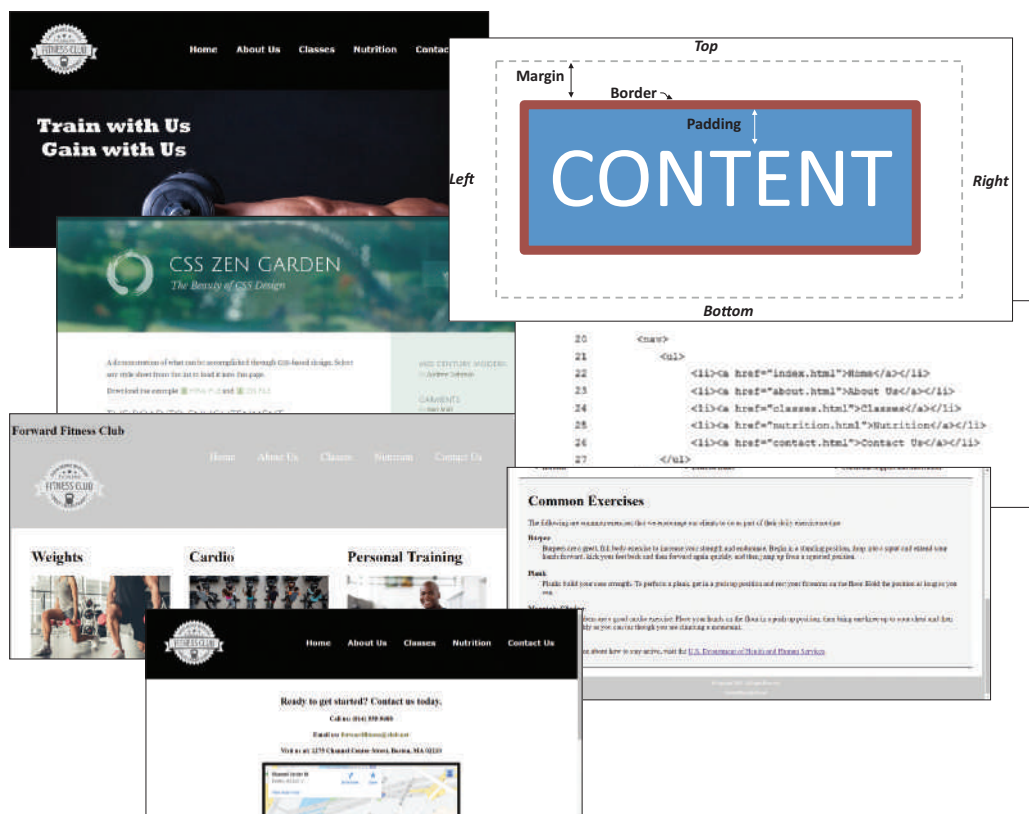


4 Designing Webpages with CSS



Objectives

You will have mastered the material in this chapter when you can:

- Explain the importance of separating design from content
- Describe Cascading Style Sheets (CSS)
- Define inline, embedded, and external styles and their order of precedence
- Describe a CSS rule and its syntax
- Explain the difference between a selector, property, and value
- Create styles that use text and color properties
- Explain the difference between inline and block content
- Describe and use the CSS box model to apply margins, padding, and borders
- Create an external style sheet and link it to an HTML page
- Create responsive images
- Use float and clear properties
- Create and style id and class attributes
- Use a span element
- Add comments to an external style sheet and validate a CSS file

4 | Designing Webpages with CSS

Introduction

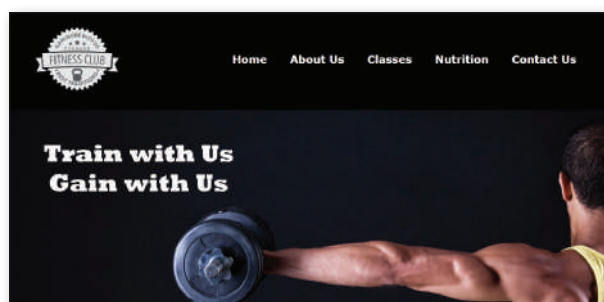
Creating a well-designed website that captures your audience's attention is vital in attracting and obtaining new customers. To do so, a web developer can use formatting such as font styles, font colors, white space, and background colors or images to increase the visual appeal of the webpages. Websites developed with HTML 5 alone can be functional, but they lack this important element of visual appeal. To improve the appearance of a website by including color, formatting text, and adding margins, borders, and shadows, for example, you need to apply styles created with **Cascading Style Sheets (CSS)**, a language you use to describe the formatting of a document written in a markup language such as HTML 5. While HTML provides the structural foundation of a webpage, you use CSS styles to determine the formatting for a webpage. By defining CSS styles in a style sheet separate from the HTML code, you can format a webpage in an unlimited number of ways, such as changing the background color of the webpage, increasing the size of text or bolding it, applying margins to a section, and adding borders to elements.

Project — Format Webpages with CSS

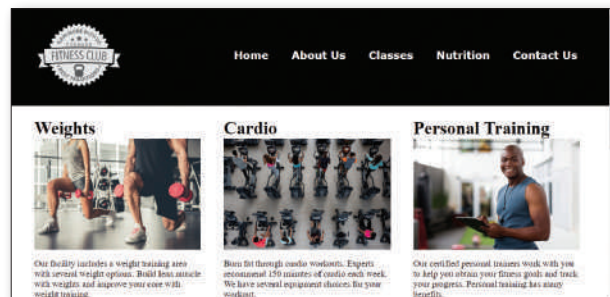
In previous chapters, you created a website template and three webpages for the Forward Fitness Club website. You also added content and links to the home page, the About Us page, and the Contact Us page. However, these pages lack formatting to enhance their appearance.

In Chapter 2, you created a website plan to guide the design and development of the Forward Fitness Club website. The plan provides information about the desired typography and colors to enhance the business brand and logo. The plan includes a wireframe, which specifies passive white space between HTML elements. To add these design elements according to your plan, you create and apply CSS styles in an external style sheet, a separate file that contains the styles for the website.

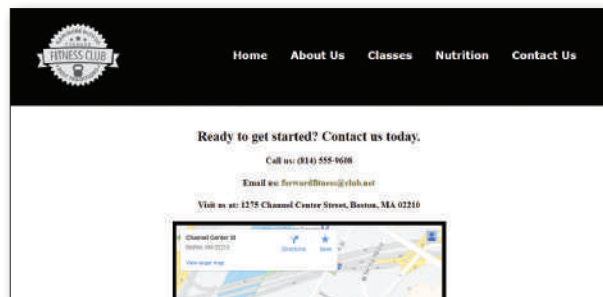
The project in this chapter enhances a website with CSS. You apply styles to the HTML 5 elements on each page to give the site a certain look and feel. After creating a style sheet, you link it to all of the pages in the website, including the template. As you add styles to the style sheet, they immediately format the attached webpages when you open or refresh the pages in a browser. Figure 4–1 shows the home page after it has been enhanced by CSS.



(a) Home Page



(b) About Us Page



(c) Contact Us Page

Figure 4-1

Roadmap

In this chapter, you will learn how to create the webpage shown in Figure 4-1. The following roadmap identifies general activities you will perform as you progress through this chapter:

1. **CREATE** a **CSS FILE**.
2. **LINK HTML PAGES TO** a **CSS FILE**.
3. **CREATE STYLE RULES**.
4. **ADD COMMENTS** to the CSS file.
5. **VALIDATE** the **CSS FILE**.

At the beginning of step instructions throughout the chapter, you will see an abbreviated form of this roadmap. The abbreviated roadmap uses colors to indicate chapter progress: gray means the chapter is beyond that activity; blue means the task being shown is covered in that activity; and black means that activity is yet to be covered. For example, the following abbreviated roadmap indicates the chapter would be showing a task in the 4 **ADD COMMENTS** activity.

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 CREATE STYLE RULES
4 **ADD COMMENTS** | 5 VALIDATE CSS FILE

Use the abbreviated roadmap as a progress guide while you read or step through the instructions in this chapter.

Using Cascading Style Sheets

Although HTML allows web developers to make changes to the structure and content of a webpage, HTML is limited in its ability to define the appearance of one or more webpages. Instead, you use CSS styles to position and format elements on a webpage.

A **style rule**, or **rule-set**, is a rule that defines the appearance of an element on a webpage. You can include CSS styles in a section of an HTML document or in a separate file. In either case, the set of CSS style rules is called a **style sheet**.

Style sheets provide a means to separate style from content. This is ideal for you as a web designer because it gives you the flexibility to easily redesign or rebrand a website. For example, visit the CSS Zen Garden at www.csszengarden.com (Figure 4–2). This website displays an array of styles, all applied to the same HTML page. Review the styles to see how each one changes the webpage. Note that the text content does not change, only the graphics and the styles as defined in the style sheet. If you want the ability to easily redesign a website, keep your content separate from the style.

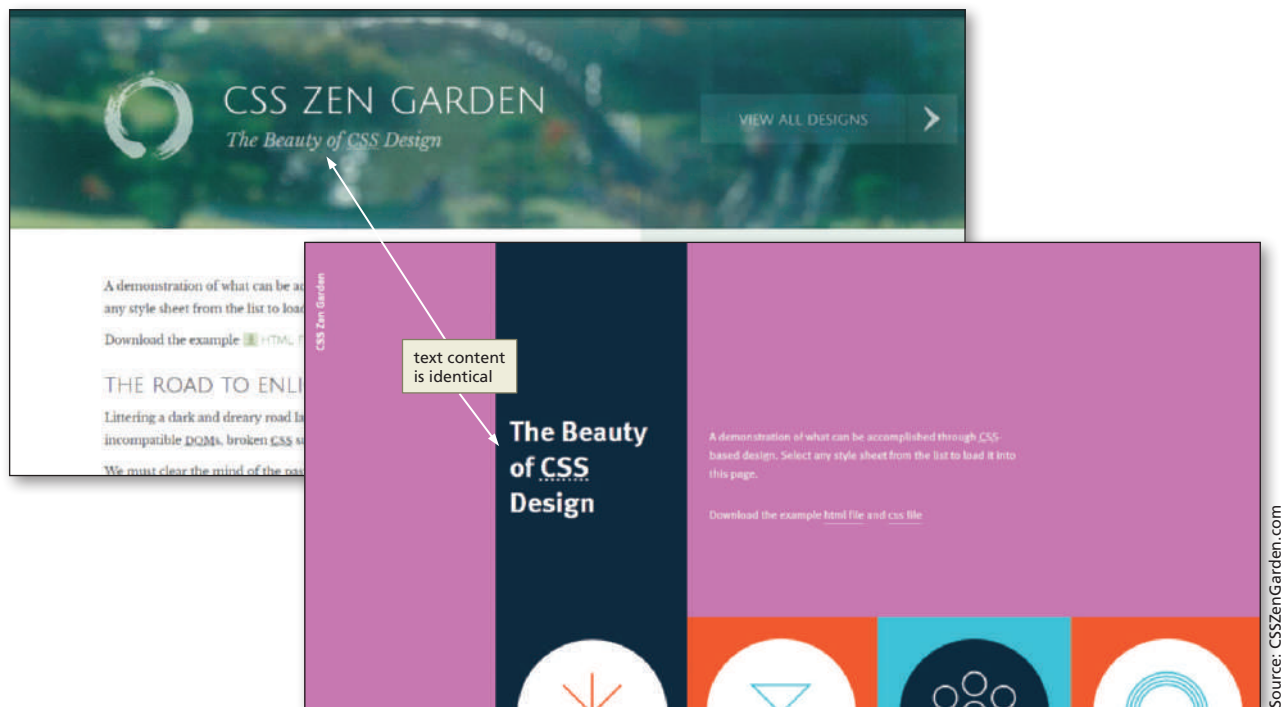


Figure 4–2

Separating content from style is even more important when you are maintaining a large website. You can define styles in a single CSS style sheet file, and then attach the single file to several webpages to apply the styles to all the attached pages. If you decide to use a different background color on all the webpages in the site, for example, you can make one change to the CSS style sheet to automatically update every page. Not only does this save a lot of time, it also means less maintenance for you or another web developer. If the site contained styles in the HTML code, locating those styles and removing them could be a time-consuming process.

CSS was developed by the W3C and is still evolving. The first version of CSS, CSS 1, was released in 1996. This version of CSS included styles for text, color, alignment, borders, padding, and margins. The second version of CSS, CSS 2, was released two years later in 1998 and included new styles to control the positioning of elements. The third version of CSS, CSS3, was released in 2001. It added many new style features, including shadows, rounded borders, and enhanced text effects.

The CSS Working Group oversees CSS specifications. Instead of releasing new CSS versions, the CSS Working Group now sets standards for CSS levels. The group releases CSS Snapshots, which include updates and revisions to CSS Levels. Visit w3.org/Style/CSS/ for the latest updates to CSS.

Modern browsers support CSS 1 and 2 and many CSS3 features. However, browsers may vary in how they apply CSS styles, so be sure to test and view your site in all the major browsers. Visit www.quirksmode.org to learn which browsers support each CSS style.

The W3C developed CSS as the primary way to format webpages, so you should not use HTML tags to style page content. Tags, such as the `<font>` tag, have been deemed as obsolete by W3C and should not be used.

Inline Styles

CSS supports inline, embedded (or internal), and external styles. With an **inline style**, you add a style to the start tag for an element, such as a heading or paragraph, using the **style attribute**. The style changes the content marked up by a specific pair of tags, but does not affect other content in the document. Because inline styles take precedence over other types of styles and affect the style for individual pieces of content, they are helpful when you need to format only one section of a webpage in a unique way. However, inline styles defeat the purpose and advantages of separating style from content, so they should be used sparingly. An example of an inline style is shown in Figure 4-3. The style rule applies only to the content in this **h1** element, the “Special Note” text. In the starting `<h1>` tag, you begin writing an inline style using the **style** attribute. The inline style shown in Figure 4-3 defines the font color of the “Special Note” text as navy blue.

BTW

Deprecated and Unsupported Elements

A **deprecated** element or attribute is one that is outdated. Deprecated elements may become obsolete in the future, though most browsers continue to support deprecated elements for backward compatibility. In addition, many tags and attributes are not supported by HTML 5, as noted in Appendix A. You can also visit w3c.org to find a list of deprecated HTML tags.

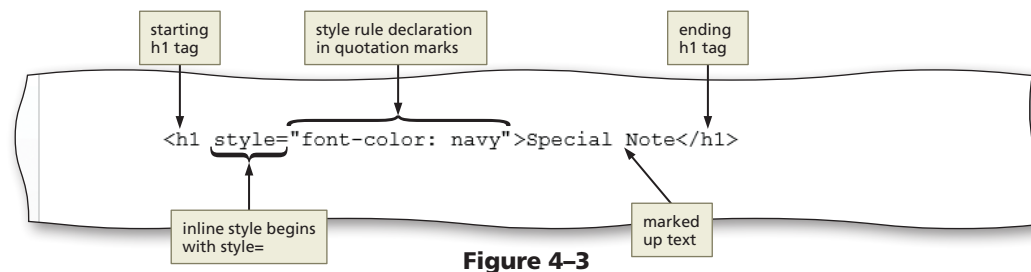


Figure 4-3

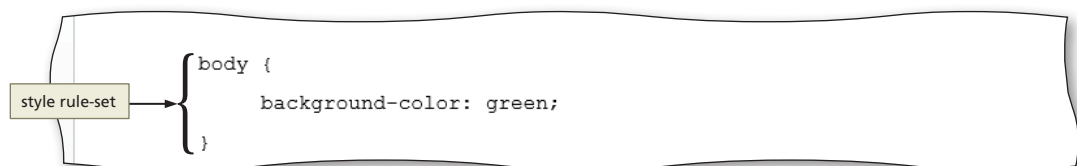
Embedded Style Sheets

An **embedded style sheet**, also called an **internal style sheet**, includes the style sheet within the opening `<head>` and closing `</head>` tags of the HTML document. Use an embedded style sheet when you want to create styles for a single webpage that are different from the rest of the website. An embedded style sheet takes precedence over an external style sheet. An example of an embedded style sheet is shown in Figure 4-4. In embedded style sheets, you place the style rules between the opening `<style>` and closing `</style>` tags. The style rule shown in Figure 4-4 sets the background color for the body section to green for the current webpage only.

**Figure 4-4**

External Style Sheets

An **external style sheet**, also called a **linked style sheet**, is a CSS file that contains all of the styles you want to apply to more than one page in the website. An external style sheet is a text file with the **.css** file extension. To apply an external style sheet, you link it (or attach it) to a webpage using a link tag in the head section of the webpage. External style sheets give you the most flexibility to quickly change webpage formats because the styles in an external style sheet are applied to every page linked to that style sheet. Changing the look of an entire website is sometimes called **reskinning** the website. You will create an external style sheet for the Forward Fitness Club's website to enhance the appearance of the **body**, **header**, **nav**, **main**, and **footer** content. An example of a style rule for an external style sheet is shown in Figure 4-5. Like the style rule shown in Figure 4-4, this one sets the background color for the body section to green. Note that external style sheets do not contain any HTML tags.

**Figure 4-5**

Style Sheet Precedence

Style sheets are said to “cascade” because each type of style has a specified level of precedence (or priority) in relationship to the others. For example, suppose you create an inline style to change the text color of an **h1** heading to red. In the **head** section of a webpage, you also create a style in an embedded style sheet to change **h1** headings to blue, and in an external style sheet, you create a style to change **h1** headings to green. What color would **h1** headings be in the webpage with the inline style? They would be red because the style closest to the content takes precedence. In other words, inline styles beat embedded styles, and embedded styles beat external styles.

CSS properties can be inherited from a parent element. This principle is called **inheritance**. For example, paragraphs and headings inherit the font and color rules for the body selector. If a selector has more than one CSS rule, **specificity** determines which CSS rule to apply. The more specific selector is applied. For example, if the value of the background-color property for the body selector is green but the one for p is blue, the p elements will have a blue background because the p selector is more specific than the body selector.

The best practice is to apply inline styles when you want to control the style of content within one pair of HTML tags, an embedded style sheet when you want to change the style of one page, and external or linked style sheets for the styles that apply to many or all pages in the website.

Web designers use external style sheets for most of their styles for many reasons. First, they can create or modify all of the styles in one style sheet file and then link the file to all of the HTML documents. This provides consistency throughout the website and lessens the need for coding beyond the single style sheet. Second, when an HTML document is linked to a CSS file, the page can load in a browser quickly. Third, if web designers need to change the appearance of a particular HTML element, they can do so in one file. The browser applies the change to all of the linked pages, reducing errors and redundancies.

CSS Basics

To write an inline CSS style, you use a style attribute within the HTML element, as shown in Figure 4–3. To write a CSS style rule for an embedded or external style sheet, you write a statement that follows the CSS syntax. Each CSS style rule consists of a selector and a declaration block. See Figure 4–6.

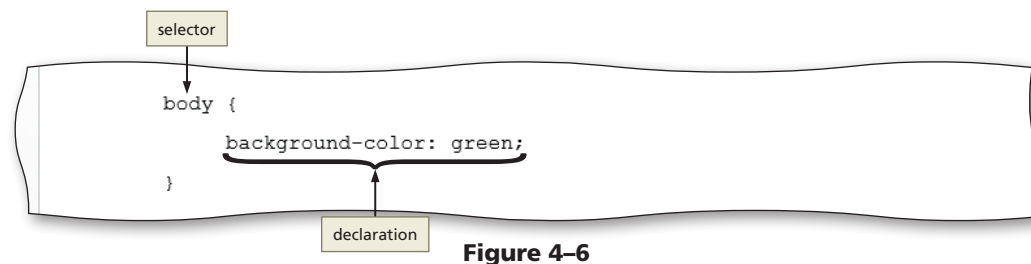


Figure 4–6

The **selector** is the part of the statement that identifies what to style. Any HTML 5 element such as `body`, `header`, `nav`, `main`, or `footer` may be a selector. For example, if you want to format the content in the `body` section of a webpage, use `body` as the selector in the style statement. A selector may also be the value of an id or class attribute. For example, if you want to format the content in the `div id="exercises"` section, use `#exercises` as the selector. (You will learn about including a # sign with selectors shortly.)

The **declaration** defines the exact formatting of the style. A declaration consists of a property and a value, separated by a colon and followed by a semicolon (;). The **property** identifies the style quality or characteristic to apply, such as color (text color), background-color, text-align, border-width, or font-size. A declaration includes at least one property to apply to the selected element.

For each property, the declaration includes a related **value** that identifies the particular property value to apply, such as green for color or 150% for font-size. You can use only certain values with each property based on the styles that property can define. The font-color property, for example, can accept navy as a value, but cannot

BTW Style Rule Syntax

The correct syntax for style rules is to use all lowercase letters for the property name without any spaces. The same is true for values: use all lowercase characters and no spaces.

BTW
CSS Syntax Color Coding in Notepad++
Notepad++ and other text editors use color coding to help you write style rules using the correct syntax. For example, when a declaration is written with correct syntax, the value is displayed in black and the property is displayed in a contrasting color by default.

accept 150% because that is not a valid value for the color property. See Appendix B for a list of properties and valid values.

Figure 4-7 shows the correct syntax for the style rule shown in Figure 4-4 and Figure 4-5. To create a style for an embedded or external style sheet, you first identify the selector, followed by a space and an opening brace. Next, you specify the property, followed by a colon, and then provide a value for the property, followed by a semicolon. Close the rule with a closing brace.

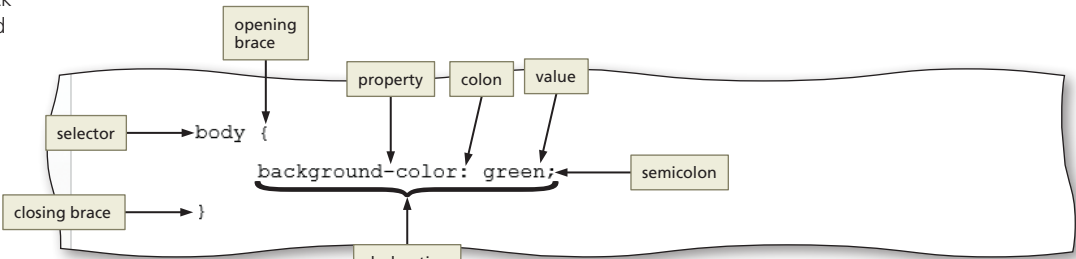


Figure 4-7

**CONSIDER THIS**

What is the difference between HTML attributes and CSS properties?

You use attributes in HTML elements to define more information for an element. Enter an attribute in an element's start tag as a name="value" pair. For example, in an `img` element, you include an attribute such as `src="logo.png"` to specify the source of the image. In this example, "src" is the attribute name and "logo.png" is the value. In a similar way, a CSS style rule defines the formatting for one or more elements. A style rule includes a selector and a declaration that consists of a property: value; pair. For example, `h1 {color: blue;}` formats `h1` elements using blue text. Enter a declaration for a selector in a style sheet.

CSS Text Properties

You can use CSS styles to format webpage text in a variety of ways. For example, use the font-family property to define a specific font. Use the font-size property to define a specific font size.

Table 4-1 lists common text properties and values.

Table 4-1 Common Text Properties and Values		
Property	Description	Common Values
font-family	Specific and general font names	font-family: Cambria, "Times New Roman", serif; font-family: Verdana, Arial, sans-serif; font-family: Georgia, "Times New Roman", serif;
font-size	Absolute or relative size of a font	font-size: 1.5em; font-size: 50%; font-size: x-large; font-size: 14pt;
font-weight	Weight of a font	font-weight: bold; font-weight: bolder; font-weight: lighter;
font-style	Style of a font	font-style: normal; font-style: italic; font-style: oblique;
text-align	Alignment of text	text-align: center; text-align: right; text-align: justify;
color	Color of text	color: red; color: blue; color: green;

Notice that the `font-family` property includes multiple values. You should provide more than one value for this property in case the browser does not support the primary font. The additional values are called **fallback values**. Specifying two or more font values is known as specifying a **font stack**. If the browser does not support the primary font, it displays the second font family indicated and if the browser does not support the second font family value, the browser uses the next font family. Commas separate each value. The desired value is listed first and the value of serif or sans-serif is listed last. For example, the declaration `font-family: Cambria, "Times New Roman", serif;` means that the browser should use the Cambria font; if the browser cannot use Cambria, it should use Times New Roman, which is listed in quotation marks because the font family name contains more than one word. Finally, if the browser does not support Cambria or Times New Roman, it should use its default serif font.

BTW

Font Names

The W3C recommends quoting font family names that contain spaces, digits, or punctuation characters other than hyphens.

Why would a browser not support certain fonts?

Fonts are installed on a computer, so a computer must have the font installed before a browser can display it. For a list of common web fonts, visit www.w3schools.com/cssref/css_websafe_fonts.asp.



CONSIDER THIS

CSS measures font size using many measurement units, including pixels, points, and ems, and by keyword or percentage. Table 4–2 lists units for measuring font size.

Table 4–2 Font Size Measurement Units

Unit	Definition	Example	Comments
em	Relative to the default font size of the element	font-size: 1.25em;	Recommended by W3C; sizes are relative to the browser's default font size
%	Relative to the default font size of the element	font-size: 50%;	Recommended by W3C; sizes are relative to the browser's default font size
px	Number of pixels	font-size: 25px;	Depends on screen resolution
pt	Number of points	font-size: 12pt;	Use for printing webpages
keyword	Relative to a limited range of sizes	font-size: xx-small;	Sizes are relative to the browser's default font size, but size options are limited

The `em` is a relative measurement unit that the W3C recommends for values of the `font-size` property. The size of an `em` is relative to the current font size of the element. For example, `1.25em` means 1.25 times the size of the current font. If a browser displays paragraphs using 16-point text by default, a `font-size` property value of `1em` is 16 points and `1.25em` is 20 points.

Percentage measurements work in a similar way. A `font-size` property value of `2em` and of `200%` appear the same when displayed in a browser. Note that no spaces appear in the `font-size` property value, so `1.25em` and `50%` are correct, but `1.25 em` is not correct.

CSS Colors

One way to help capture a webpage visitor's attention is to use color as a webpage background or for text, borders, or links. HTML uses color names or codes to designate color values. When using a color name, you can specify a word such as `aqua` or `black` as a value. Following are 16 basic color names. For more color names, visit www.w3schools.com/colors/colors_names.asp.

aqua	navy
black	olive
blue	purple
fuchsia	red
gray	silver
green	teal
lime	white
maroon	yellow

Color codes are more commonly used in web design. Three common types of color codes used with CSS are hexadecimal, RGB, and HSL.

Hexadecimal values consist of a six-digit number code that corresponds to **RGB (Red, Green, Blue)** color values. When noting color values in CSS, include a number sign (#) before the code. Hexadecimal is a combination of the base-16 numbering system, which includes letters A through F. An example of a hexadecimal color value is 0000ff, which is blue. The first two digits (00) indicate the red value, which is none in this case. The next two digits (00) indicate the green value, which is also none in this case. The last two digits (ff) indicate the blue value. Because ff is the highest two-digit hexadecimal number, 0000ff specifies a pure blue.

RGB notation is used to display colors on a screen, though not in print. RGB blends red, green, and blue color channels to create a color. (A **channel** contains the number of red, green, or blue pixels necessary to create a specified color.) Each color channel is expressed as a number, 0 through 255. For example, the color blue is expressed as rgb(0,0,255). The first number represents the red color channel. The zero represents no pixels from a color channel; in the example, color channels red and green contribute no pixels to the color. The last channel represents the blue color channel; the number 255 represents the truest form of the color channel, in this case, blue. Hexadecimal 0000ff and rgb(0,0,255) are two ways of expressing the same color, pure blue.

The HSL color value notation was introduced with CSS3. It stands for hue, saturation, and lightness. The hue is a degree (0 to 360) on the HSL color wheel. Saturation is indicated as a percentage from 0% to 100% and is used to specify a shade of gray. Lightness is also indicated as a percentage from 0% to 100%. Black is 0% and white is 100%. To use HSL, you use the hsl() function. To create the color red, for example, use the notation hsl(0, 100%, 50%). HSL is supported by modern browsers.

Table 4–3 shows a common list of colors, with the corresponding hexadecimal, RGB, and HSL color codes.

Table 4–3 Color Values			
Color	Hexadecimal	RGB	HSL
Black	#000000	rgb(0,0,0)	hsl(0, 0%, 0%)
White	#ffffff	rgb(255,255,255)	hsl(0, 0%, 100%)
Red	#ff0000	rgb(255,0,0)	hsl(0, 100%, 50%)
Green	#008000	rgb(0,128,0)	hsl(120, 100%, 25%)
Blue	#0000ff	rgb(0,0,255)	hsl(240, 100%, 50%)
Yellow	#ffff00	rgb(255,255,0)	hsl(60, 100%, 50%)
Orange	#ffa500	rgb(255,165,0)	hsl(39, 100%, 50%)
Gray	#808080	rgb(128,128,128)	hsl(0, 0%, 50%)

To use a color in a style rule declaration, use the color value as the property value. For example, to style a background color as gray, you use the background-color property with a value of #808080, as shown in the following example:

```
background-color: #808080;
```

BTW
HWB Colors
HWB (Hue, Whiteness, Blackness) is a new color format under development. It is not currently supported in HTML.

BTW
CMYK Colors
CMYK (Cyan, Magenta, Yellow, and Black) is a new color format under development. CMYK is a standard used for printing. It is not currently supported in HTML.

BTW
Opacity
RGB and HSL color formats also support opacity with the rgba() and hsla() functions.

BTW
Quick Color Picker
W3 Schools provides a quick color picker reference at www.w3schools.com/colors/colors_picker.asp. Here, you can select a color, view various shades of the color, and find the hexadecimal color value.

If a hexadecimal color value repeats the same six characters, you can shorten the value to three characters. For example, the hexadecimal color value for black is #000000, which can be shortened to #000.

Understanding Inline Elements and Block Elements

When you format webpages with CSS, you set rules that describe how the HTML elements should appear in a browser. As you create rules for the elements, review the structure of the HTML document because it plays a part in how a browser displays the element on a webpage.

HTML elements are positioned on the webpage as a block or as inline content. A **block element** appears as a block because it starts and ends with a new line, such as the main element or a paragraph element. Block elements can contain content, other block elements, and inline elements. **Inline elements** are displayed without line breaks, so they flow within the same line. Inline content always appears within block elements. Examples of inline elements include the span element () and the anchor element (<a>). You use the span element to group inline elements. To format one or more words in a paragraph, for example, include an inline style in the opening span tag that groups the words, as in the following example, which formats only “Warning” in red text.

```
<span style="color: red;">"Warning"</span>
```

An **img** element is also an inline element because it flows in the same line, although it has natural height and width properties unlike other inline elements. Figure 4–8 shows an example of an inline element in a block element.

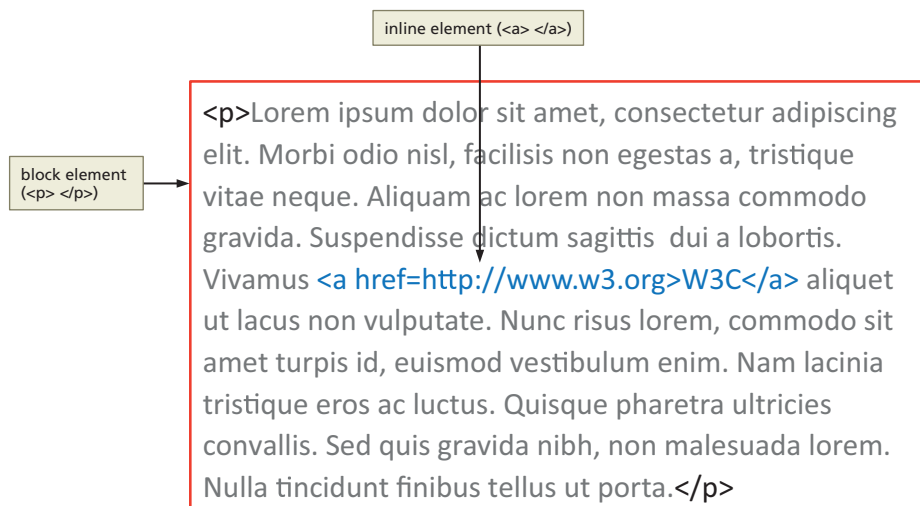


Figure 4–8

Header, nav, main, and footer are all examples of block elements. When you apply CSS styles to these block elements in the website you created for the Forward Fitness Club, you must consider their hierarchical structure to produce the visual effect you intend. Styles applied to elements below or above another element can affect their placement on a webpage. For example, as a block element, the **header** element normally starts on a new line and spans the width of the webpage. A new line also appears after the **header** element. If you want the **nav** element to appear to the right of the header, apply styles to both the **header** element and the **nav** element to accomplish this effect.

CSS Box Model

Each block element such as a header, nav, main, and footer element has a top, right, bottom, and left side. In other words, the element is displayed in a browser as a box with content. The **CSS box model** describes these boxes of content on a webpage. Each content box can have margins, borders, and padding, as shown in Figure 4–9. You refer to the sides of a box in clockwise order: top, right, bottom, and left.

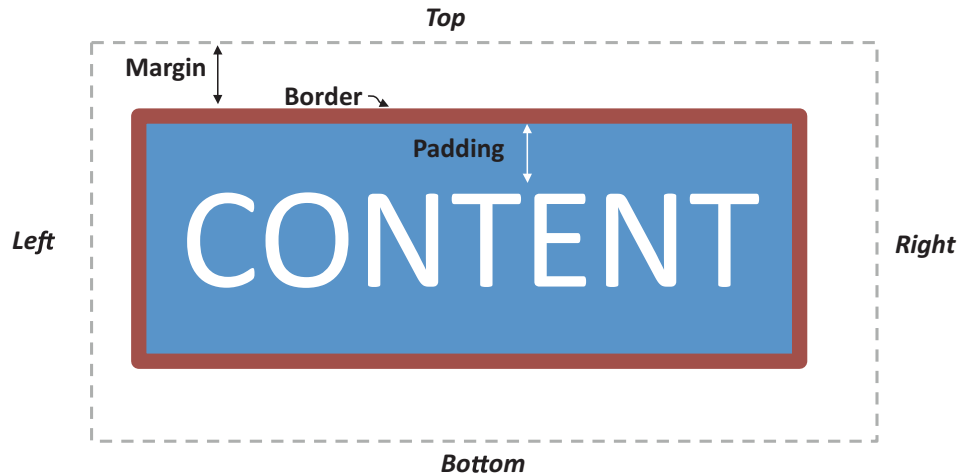


Figure 4–9

The **margin** provides passive white space between block elements or between the top or bottom of a webpage. You can define margins at the top, right, bottom, and left of a block element. Margins are transparent and are measured in ems (em), percentages (%), or pixels (px).

The **border** separates the padding and the margin of the block element. A border can vary in thickness and color and can be defined at the top, right, bottom, and left sides of a block element. A border can also have a style such as solid, dashed, or dotted.

Padding is the passive white space between the content and the border of a block element. Padding is typically measured in ems (em), percentages (%), or pixels (px). By default, padding is set to 0px, so paragraph text, for example, appears at the edges of the block element. You can increase the padding to improve legibility. The box model shown in Figure 4–9 includes more than the default amount of padding. The background color for the padding and content is always the same.

Using CSS, you can specify margin, padding, and border properties for all four sides of a block element. Following are examples of the many ways to specify block element properties.

When you want to use the same amount of margin on all four sides of a block element, specify a single value to the margin property. The following declaration specifies a margin of 2em on the top, bottom, left, and right:

```
margin: 2em;
```

You can also specify different margins for the sides. The following declarations specify a left margin of 2em and a bottom margin of 1em:

```
margin-left: 2em;  
margin-bottom: 1em;
```

You can set the top and bottom margins to one value and then set the left and right margins to another value. You do so by including two values in the declaration.

The following shorthand notation sets the top and bottom margins to 1em and the left and right margins to 2em:

```
margin: 1em 2em;
```

You can also use shorthand notation to specify three values for margin. The first value sets the top margin, the second value sets the left and right margins, and the third value sets the bottom margin. The following example sets the top margin to 2em, left and right margins to 1.5em, and bottom margin to 1em:

```
margin: 2em 1.5em 1em;
```

To specify a different value for each side, use shorthand notation to include four values within the declaration. The values move clockwise around the block element. The first value is the top, the second value is the right, the third value is the bottom, and the fourth value is the left. For example, the following declaration specifies a top margin of 1em, right margin of 2em, bottom margin of 1.5em, and left margin of 3em:

```
margin: 1em 2em 1.5em 3em;
```

You can use a similar shorthand notation to set the style, width, and color for all four sides of a block element's border. The property values can appear in any order. The following declaration sets the top, right, bottom, and left borders of a block element to a solid line using a width of 1 pixel and a color of black:

```
border: solid 1px #000000;
```

You can set even more specific property options for the border, such as border-top-width and border-right-color. For example, the following declaration sets only the left border of the block element to a width of 2 pixels:

```
border-left-width: 2px;
```

Table 4–4 lists common CSS box model properties used to style block elements.

Table 4–4 Common CSS Box Model Properties

Property	Description	Examples
margin	Sets the amount of space around the block element (top, right, bottom, left)	margin-top: 2em; margin-bottom: 15%; margin: 20px;
padding	Sets the amount of space between content and the border of its block element	padding-left: 1.5em; padding-right: 12%; padding: 10px;
border	Sets the format of the block element's border	border: solid 1px #000000;
border-style	Designates the style of a border	border-top-style: solid; border-top-style: dotted;
border-width	Designates the width of a border	border-top-width: 1px; border-bottom-width: thick;
border-color	Designates the border color	border-top-color: #000000; border-bottom-color: gray;
border-radius	Rounds the corners of a block element's border	border-radius: 10px;
box-shadow	Adds a shadow to a block element's border	box-shadow: 8px 8px 8px #000000;

To have a border appear around the content in a block element, you must specify a border style in a CSS statement. You can include the style value with the border property or use the border-style property with an assigned value.

The border-radius and box-shadow properties are CSS3 properties. As with the other border properties, you can list more than one value to set the top, right, bottom, and left radius values for the border. For the box-shadow property, you must specify the horizontal and vertical offset measurements. In addition, you can set the distance of the shadow's blur, the size of the shadow, and its color. The default color is black.

BTW

CSS Box Properties

Appendix B contains a comprehensive list of the CSS box properties and their acceptable values.

Creating an External Style Sheet

To create style rules that apply to more than one webpage in a website, use an external style sheet. Recall that an external style sheet is a text file that contains the style rules you want to apply to more than one page in the website.

Using an external style sheet involves two steps. First, use a text editor to create and save a document with a .css extension. In the CSS document, create style rules for elements such as body, header, nav, main, and footer to improve the visual appeal of the website. Create rule-sets to add color and borders, apply text properties, align items, and increase white space for padding and margins. Next, link the CSS file to the webpages that should be formatted using the styles defined in the external style sheet.

Selectors

Recall that a style rule begins with a selector, which specifies the element to style. A selector can be an HTML element name, an id attribute value, or a class attribute value. If the selector is an HTML element, you use the element name for the selector. Figure 4–10 shows how a selector in a CSS file selects the content to be styled for an HTML element, an id attribute, and a class attribute for HTML elements in an HTML file.

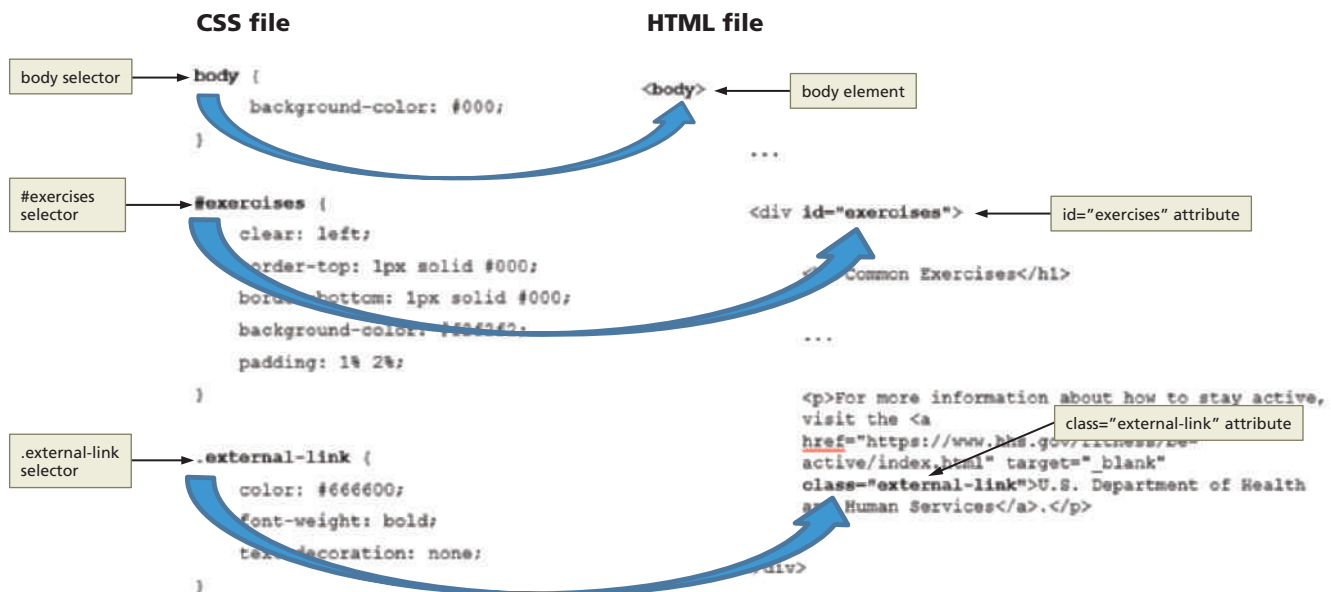


Figure 4–10

For example, to style the header element, use **header** as the name of the selector. To define the style of all p elements so they display white text on a black background, use the following style rule:

```

p {
    color: white;
    background-color: black;
}

```

If you want to apply styles to some p elements but not others, for example, you can create an id or a class selector. An **id selector** uses the id attribute value of an HTML element to select a single element. To create an id selector, you begin the style rule with a number sign (#) followed by the id attribute value. For example, to style the `div id="container"` element, use `#container` as the selector. The following style rule applies a solid 2-pixel border to only the `div id="container"` element:

```
#container {
    border: solid 2px;
}
```

Use a **class selector** to select elements that include a certain class attribute. To create a class selector, you begin the style rule with a period (.) followed by the class attribute value. You will learn more about class selectors later in this chapter. For example, to style `class="mobile"`, use `.mobile` as the selector. The following style rule sets the font size to 1em for all elements that use mobile as their class attribute value:

```
.mobile {
    font-size: 1em;
}
```

If you want to create a style rule that applies to more than one element, you can list more than one selector. For example, the following style rule sets the same font family property and values for the `header`, `nav`, and `footer` elements:

```
header, nav, footer {
    font-family: Calibri, Arial, sans-serif;
}
```

If you want to create a style rule that applies to an element contained within another element, you list the elements in the order they appear to create the selector in the style rule. This type of selector is called a **descendant selector**. For example, the following style rule sets the list-style property to none for list items in an unordered list included in the navigation area:

```
nav ul li {
    list-style: none;
}
```

The following example is another way to create a style rule that selects all the li elements within the nav parent element:

```
nav > li {
    font-size: 1.5em;
}
```

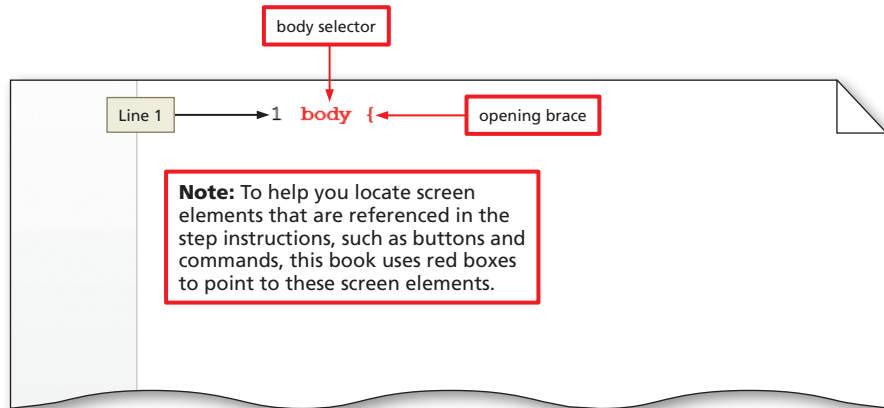
To Create a CSS File and a Style Rule for the Body Element

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 CREATE STYLE RULES
4 ADD COMMENTS | 5 VALIDATE CSS FILE

To apply styles to the Forward Fitness Club website, you can create them in an external style sheet. *Why? The styles defined in the external style sheet will apply to all the webpages in the website.* To create a CSS file, you use your preferred text editor to create a file and save it as a CSS document. Next, include one or more style rules that apply to all the webpages linked to the CSS document. The first style rule you add sets the background color of the webpages. This rule uses “body” as the selector and the background-color property in the declaration. Because the `body` element contains all of the content displayed on a webpage, using “body” as the selector sets the background color of the entire webpage. The following steps create a CSS file and a style rule.

1

- Open your text editor, tap or click File on the menu bar, and then tap or click New if you need to open a new blank document.
- Tap or click File on the menu bar and then tap or click Save As to display the Save As dialog box.
- Navigate to your fitness folder and then double-tap or double-click the css folder to open it.
- In the File name box, type **styles** to name the file.

**Figure 4-11**

Q&A Is “styles” the required file name for the CSS file?

No. You can give a CSS file any meaningful name, but a common name for an external style sheet is styles.css. An external style sheet must end with a .css extension.

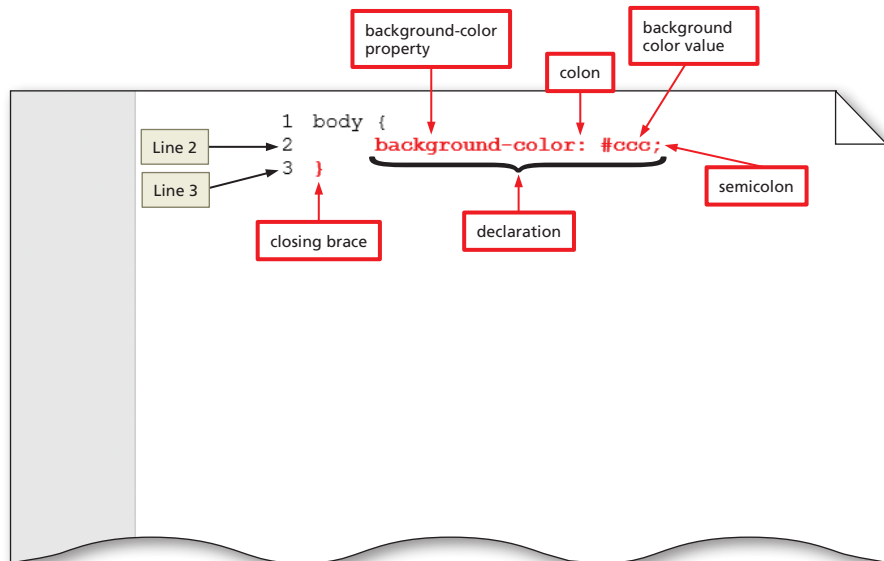
- Tap or click the Save as type button, and then tap or click Cascading Style Sheets or CSS to select the file format.
- Tap or click the Save button to save the file in the css folder.
- On Line 1 of the text editor, type **body {** to begin a new style rule for the body element (Figure 4-11).

Q&A Why should I use “body” as the selector?

Using “body” as the selector means the style rule applies to all of the body elements in the webpages linked to the style sheet. In this case, you are creating a style rule that sets the background color for the entire webpage because the body element contains all of the content displayed on a webpage.

2

- Press the ENTER key to add Line 2, press the TAB key to indent the new line, and then type **background-color: #ccc;** to add a declaration that sets the background color.
- Press the ENTER key to add Line 3, press the SHIFT+TAB keys to decrease the indent, and then type **}** to add a closing brace (Figure 4-12).

**Figure 4-12**

Q&A What color does ccc represent?

The color specified by ccc is a light shade of gray. The hexadecimal color code for this shade of gray is cccccc. Because it consists of six "c's," the color code can be shortened to ccc.

3

- Save your changes.

Linking an HTML Document to a CSS File

After creating a CSS file, link it to all the webpages that will use its styles. Otherwise, the browser will not find and apply the styles in the external style sheet. Insert a **link** element on the HTML page within the `<head>` and `</head>` tags, which is the section of an HTML document that provides information to browsers and search engines but is not displayed on the webpage itself.

The **link** element uses two attributes, **rel** and **href**. The **rel** attribute uses the **stylesheet** value to indicate that the document is linked to a style sheet. The **href** attribute value specifies the file path or file name of the CSS file. Following is an example of a link to a style sheet named `styles.css` and stored in the `css` folder:

```
<link rel="stylesheet" href="css/styles.css">
```

The **type="text/css"** attribute and value is also commonly used within a **link** element to reference a CSS file. However, with HTML 5, the **type** attribute is not necessary. The W3C says that the use of the **type** attribute within the **link** element is “purely advisory.”

To Link HTML Pages to the CSS File

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 CREATE STYLE RULES
4 ADD COMMENTS | 5 VALIDATE CSS FILE

To link the `styles.css` file to an HTML page, you include a link to the style sheet within the **head** section of the document. Next, view the changes to the webpage in a browser. *Why? To apply the CSS style rules to a webpage, you must link the webpage to the CSS file. To view the applied styles, open the home page in a browser.* If you link the website template to the style sheet file, any new pages created from the template will already be linked to the style sheet. The following steps link the home, about, contact, and template HTML documents to the external style sheet file and then view one page in a browser.

- 1
 - Open `index.html` in your text editor to prepare to link the page to the CSS file.
 - Place the insertion point after the meta element on Line 6 and press the ENTER key to create a new Line 7.
 - If necessary, press the TAB key to indent the line and then type `<link rel="stylesheet" href="css/styles.css">` to create a link to the style sheet file (Figure 4-13).



Figure 4-13

Q&A Why do I need to include `css/` before the name of the file?

The `styles.css` file is located within the `css` folder. Because the file is not stored in the same folder as the `index.html` document, you must include the folder name as well as the file name in the **href** attribute value.

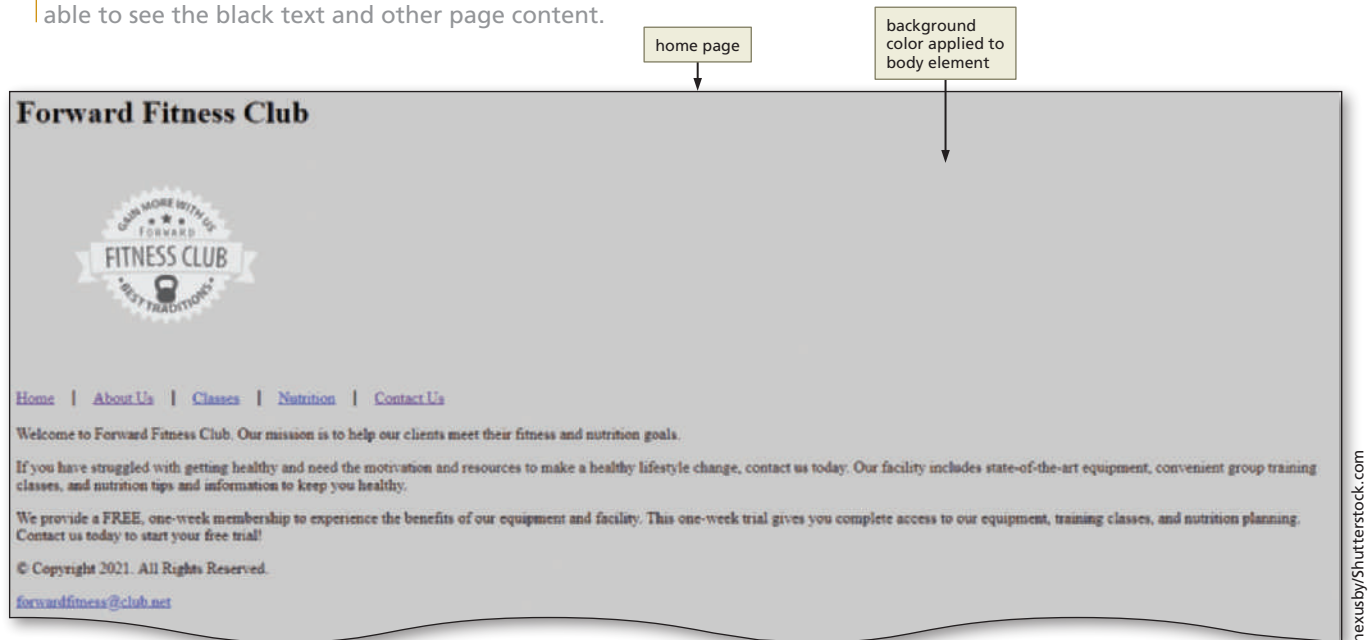
Can I place the link to the style sheet anywhere within the head element or do I have to place it directly after the meta element?

The link to the style sheet can be anywhere within the head element, though placing it below the title element is a best practice.

2

- Save the index.html page to preserve your changes.
- Open about.html in your text editor to prepare to link the page to the CSS file.
- Place the insertion point after the meta element on Line 6 and press the ENTER key to create a new Line 7.
- If necessary, press the TAB key to indent the line and then type `<link rel="stylesheet" href="css/styles.css">` to create a link to the style sheet file.
- Save your changes and close the file.
- Open contact.html in your text editor, place the insertion point after the meta element on Line 6, and then press the ENTER key to create a new Line 7.
- If necessary, press the TAB key and then type `<link rel="stylesheet" href="css/styles.css">` to create a link to the style sheet file.
- Save your changes and close the file.
- Open template.html, place the insertion point after the meta element on Line 6, and then press the ENTER key to create a new Line 7.
- If necessary, press the TAB key and then type `<link rel="stylesheet" href="css/styles.css">` to create a link to the style sheet file.
- Save your changes and close the file.
- View the home page in your default browser to view the page with the linked style sheet (Figure 4–14).

Q&A Why is the background color gray? The example shown at the beginning of the chapter shows a black background. You will change the background to black in future steps. If you use black for the background now, you will not be able to see the black text and other page content.

**Figure 4–14****CONSIDER THIS**

How can I confirm that my style rules have been correctly applied to a webpage?

After creating a CSS file, link it to one of your webpages. Include a complete style rule or add a declaration to a selector, save your changes, and then view the webpage in a browser to view the effects of the new or modified style. If the style is not applied as you intended, return to your CSS file to check for syntax errors, confirm that you saved the CSS file, and check for value errors. It is much easier to find mistakes when you code and test each new style or declaration.

Creating a Webpage Layout

To design a webpage to look the way you want, you cannot use HTML alone. You use CSS to design a webpage layout. With CSS, you can specify where each element should be placed on a webpage. Likewise, you specify how content will appear on each webpage. In Chapters 1 and 2, you learned how to create a website plan and a wireframe. Recall that a wireframe is a sketch of the proposed webpage layout. Figure 4–15 shows the proposed wireframe for the Forward Fitness Club home page.

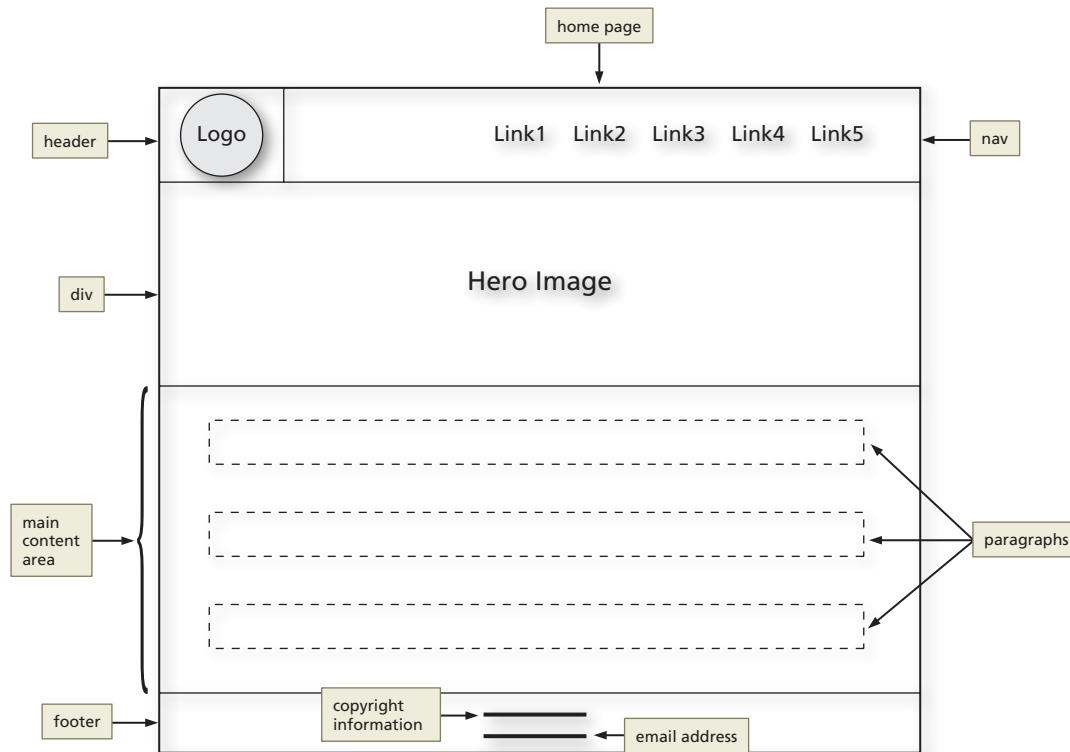


Figure 4–15

To achieve this design, you will create several style rules and use various CSS properties.

Float and Clear Properties

One way to create the layout shown in Figure 4–15 is to use the CSS float property. The **float property** allows you to position an element to the right or left of other elements, while remaining within its parent element. The float property has four valid values: **left**, **right**, **inherit**, and **none**. The **left** value floats an element left. The **right** value floats an element right. The **inherit** value floats an element with the same value as its parent element. The **none** value does not float an element and is the default.

To achieve the layout shown in the wireframe (Figure 4–15), float the header element left and float the nav element right. You must also specify a width for each

element. Create the following style rules to design the header and nav layout shown in the wireframe:

```
header {
    float: left;
    width: 25%;
}

nav {
    float: right;
    margin: 5em 3em 0 0;
    width: 70%;
}
```

The above style rule for the header floats the header to the left of the nav element and sets its width to 25%. The nav style rule floats the nav to the right of the header; sets its top margin to 5em, right margin to 3em, and top and bottom margins to 0; and specifies a width of 70%. Note the specified widths of each element: 25% and 70%. When added together, their combined width value is 95%. Because these elements share the same vertical space, their combined widths should not exceed 100%. It seems the page allows another 5% to work with; however, it does not. The nav element also includes a right margin value of 3em. Since both margin and padding values take up vertical space within that 100% allotment, you must adjust widths accordingly, so that, all together, width values, padding values, and margin values do not surpass 100%.

If you float an element, neighboring elements will also be affected by the float. Subsequently, the neighboring elements may also appear to float or wrap alongside the element with the specified float. To correct this issue, use the **clear property** to remove the float effect. The valid values for the clear property are **right**, **left**, and **both**. To clear an element that is floating left, use the **clear: left;** declaration. Likewise, to clear an element that is floating right, use the **clear: right;** declaration. Use the **both** value if elements are floating on both the right and left sides of the text. Apply the clear property to the selector that is affected by the floating element. For example, if you want the main element to appear below the header and nav, create a style rule for the main and use the clear property in one of its declarations.

```
main {
    clear: left;
}
```

When designing layouts such as columns, content can “spill over” to another element and appear outside its parent container. The effect is similar to a cup filling with water that begins to overflow from the cup. To correct this issue, you use the **CSS overflow property** to keep the content within its parent container.

Aligning Webpage Content

One way to align webpage content is to use the text-align property, which applies to block elements. Use the text-align property to set the horizontal alignment for the lines of text in an element. The text-align property can use one of four values: **left** (the default), **center**, **right**, or **justify**. Use the justify value to align

text on the left and right margins by adding white space to the lines of text. Use the center value to center content within an element. For example, the following rule centers an `h1` element:

```
h1 {
    text-align: center;
}
```

Another way to center webpage content is to use the margin property. To center all of the elements so that the page appears centered within a browser window, you create styles to set the left and right margins to auto. You also set the width property to a percentage to specify how much of the page to use for content. For example, if you set the width to 80%, you leave 20 percent of the page for margins. Using “auto” as the property value means that the left and right margins split the available 20 percent equally, leaving 10 percent of the page for the left margin and 10 percent of the page for the right margin. The following declaration specifies the width of the `div id="container"` section as 80 percent of the page and sets the left and right margins to “auto,” centering all of the content in that section:

```
container {
    width: 80%;
    margin: 0 auto;
}
```

To Position Elements

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 CREATE STYLE RULES

4 ADD COMMENTS | 5 VALIDATE CSS FILE

Now that you have created a style for the `body` element, return to your CSS file to create style rules for the header, nav, and main elements. To create the layout shown in Figure 4–15, you specify a width of 25% for the header and float it to the left. You specify a width of 70% for the nav and float it to the right. Additionally, you specify a top margin of 5em, right margin of 3em, and bottom and left margins of 0. Finally, you clear the float to the left of the main element. The following steps create style rules for the header, nav, and main to lay out the page elements.

1

- In the text editor, return to the `styles.css` file.
- Place the insertion point after the closing brace on Line 3 and press the ENTER key twice to insert new Lines 4 and 5.
- On Line 5, type `header {` to add the header selector and an opening brace (Figure 4–16).

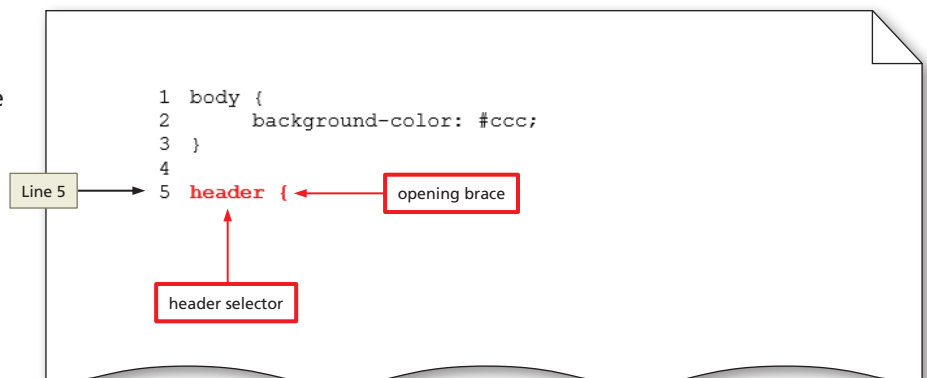
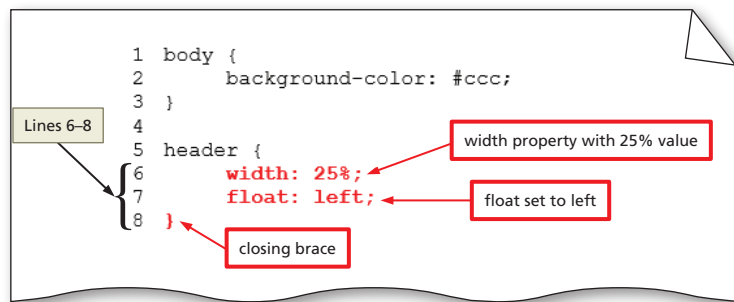


Figure 4–16

Q&A Why is Line 4 blank?
Reading and reviewing several lines of code can be hard on the eyes. For improved readability, insert blank lines between each style rule.

2

- Press the ENTER key to add Line 6, press the TAB key to indent the line, and then type **width: 25%;** to add a declaration that sets the width of the element.
- Press the ENTER key to add Line 7 and then type **float: left;** to add a declaration that sets a left float.
- Press the ENTER key to add Line 8, press the SHIFT+TAB keys to decrease the indent, and then type **}** to close the style rule (Figure 4-17).

**Figure 4-17**

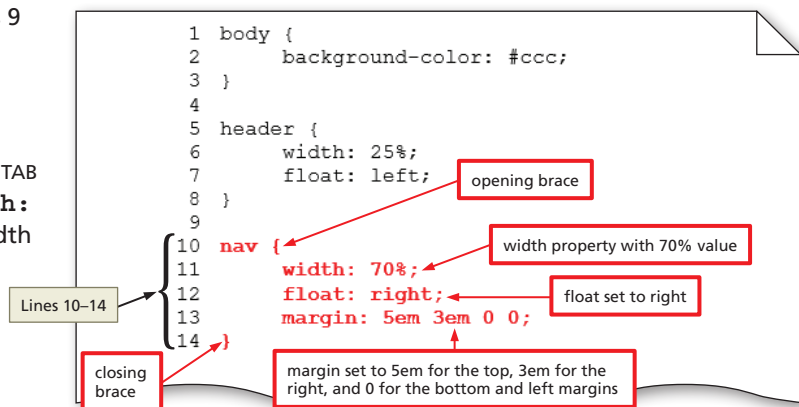
Q&A Why do I need to set the width to 25%?
 Currently, the header element on the webpage spans the entire width of the browser window. Setting the width to 25% restricts the width of the header element, allowing room for the nav element to be positioned to the right of the header element.

Do I have to enter each property and value on a separate line?

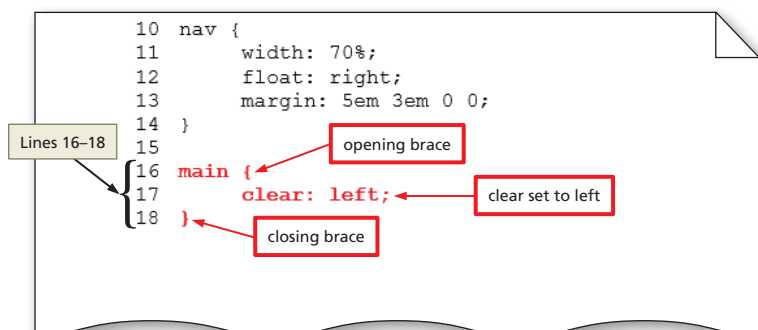
No. However, listing each property and value on separate lines enhances the readability of the style sheet. A style rule with many declarations is difficult to read unless the declarations are listed on separate lines. Readability helps to find and correct syntax errors.

3

- Press the ENTER key twice to insert new Lines 9 and 10.
- On Line 10, type **nav {** to add the nav selector and an opening brace.
- Press the ENTER key to add Line 11, press the TAB key to indent the line, and then type **width: 70%;** to add a declaration that sets the width of the element.
- Press the ENTER key to add Line 12, and then type **float: right;** to set a right float.
- Press the ENTER key to add Line 13, and then type **margin: 5em 3em 0 0;** to set the margin on all sides of the element.
- Press the ENTER key to add Line 14, press the SHIFT+TAB keys to decrease the indent, and then type **}** to close the style rule (Figure 4-18).

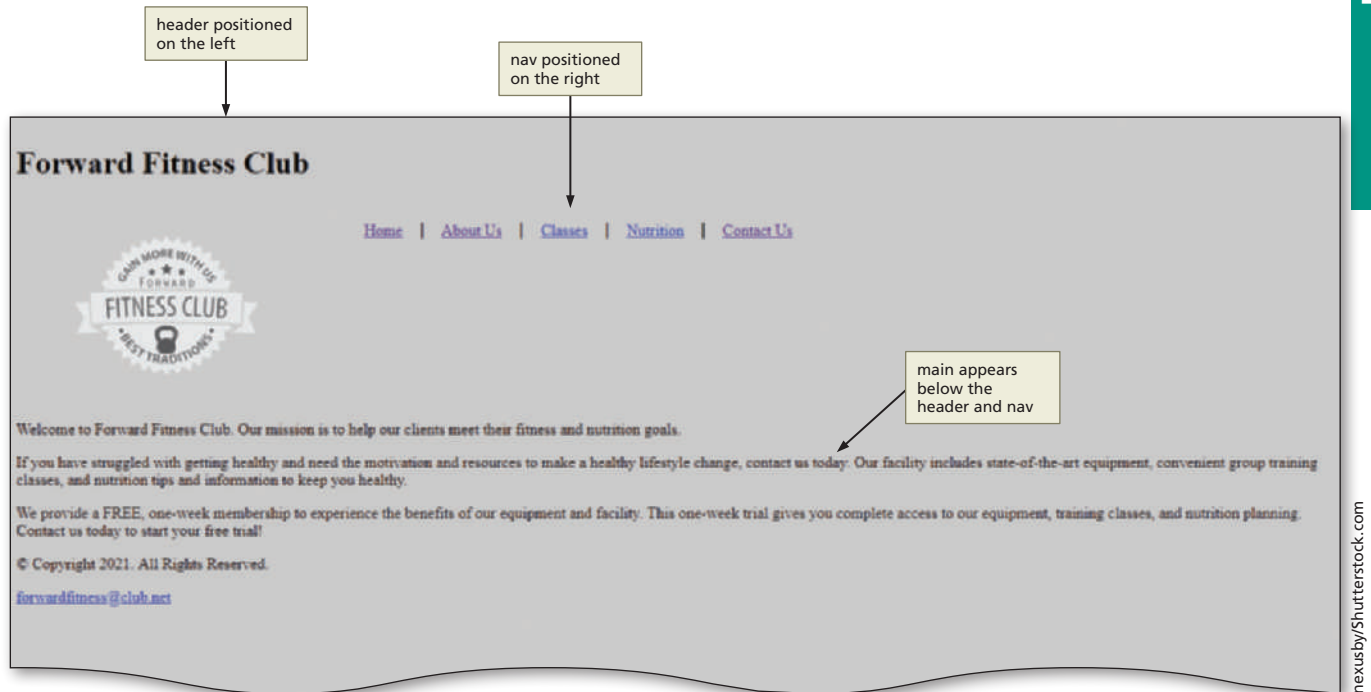
**Figure 4-18****4**

- Press the ENTER key twice to insert new Lines 15 and 16.
- On Line 16, type **main {** to add the main selector and an opening brace.
- Press the ENTER key to add Line 17, press the TAB key to indent the line, and then type **clear: left;** to add a declaration that clears a float to the left.
- Press the ENTER key to add Line 18, press the SHIFT+TAB keys to decrease the indent, and then type **}** to close the style rule (Figure 4-19).

**Figure 4-19**

5

- Save your changes.
- View the home page in a browser to see the applied styles (Figure 4–20).

**Figure 4–20**

Creating Style Rules for Structural Elements

The next formatting task is to create style rules for the HTML 5 structural elements used in the Forward Fitness Center website. Because the header section appears at the top of the webpage, it needs formatting that makes the header contents stand out and attract visitors to the page. You have already completed this task by creating a style rule that sets width and float properties.

The nav section should also be prominent and easy to find on the webpage, so you should format it differently from the other structural elements. Update the nav to use an unordered list, and then create several style rules to format the nav to appear horizontally on the page and remove the bullet markers from each list item.

Because a float was specified for the header and nav, you will need to use the `clear` property within the main style rule. This will keep the position of the main element below the nav element. Each page contains different content within the main element, so you will create various style rules to style the content for each page.

Finally, create a style rule that formats the footer section by defining the font size, text alignment, and padding of the `footer` element.

BTW

CSS Style Rule Order

Although the styles in a style sheet are not required to follow a specific order, most designers list the styles in the same general order that they will be used on a webpage. In other words, styles that apply to the body, outer container, and sections of the wireframe are typically listed first. Styles that apply to more specific areas of content such as headings, lists, and links are generally listed next.

Use a List for Navigation Links

To make the navigation links more appealing on the webpage, you can format them with CSS. For the Forward Fitness Club website, you will code the navigation links as an unordered list in the HTML document and then add styles in the CSS file to better display and position the navigation system as appropriate for your design.

Format the navigation links by increasing the font size and changing the text color to white, which will create a strong color contrast for the Forward Fitness Club website. Use CSS to create extra space between links to clearly separate the links.

To accomplish this design, first code the navigation links as an unordered list. This is a common web design technique that both semantically describes the content and lets you format each link using CSS. Next, add styles to the style sheet to format the navigation list items without displaying a bullet. You will also apply a background color, text color, and a rounded border to make each link look like a button. You will then remove the underlining that appears with links by default.

To Code the Navigation Links as an Unordered List

Code the navigation links on the home, About Us, Contact Us, and template pages as list items in an unordered list. *Why? Coding links in a navigation system as an unordered list is a common way to semantically describe the content because a list of links is truly an unordered list. This technique also provides the structure, or “box,” around each link, which is required to style the links in different ways for different viewport sizes later.* The following steps add the HTML code to mark up the links in the nav section as list items in an unordered list.

- 1
 - Open the index.html file in your text editor.
 - Replace the beginning paragraph tag `<p>` on Line 21 with a beginning unordered list tag, `<ul>`, to insert an unordered list.
 - Place your insertion point after the `<ul>` tag and press the ENTER key to place the first anchor element on a new Line 22.
 - Starting on Line 22, add the li elements and closing ul tag as shown in Figure 4–21 to code the navigation links as list items in an unordered list.

Q&A Do I need to indent the lines of code as I insert the tags?
Yes. Be sure to indent as shown in Figure 4–21 to maintain maximum readability.

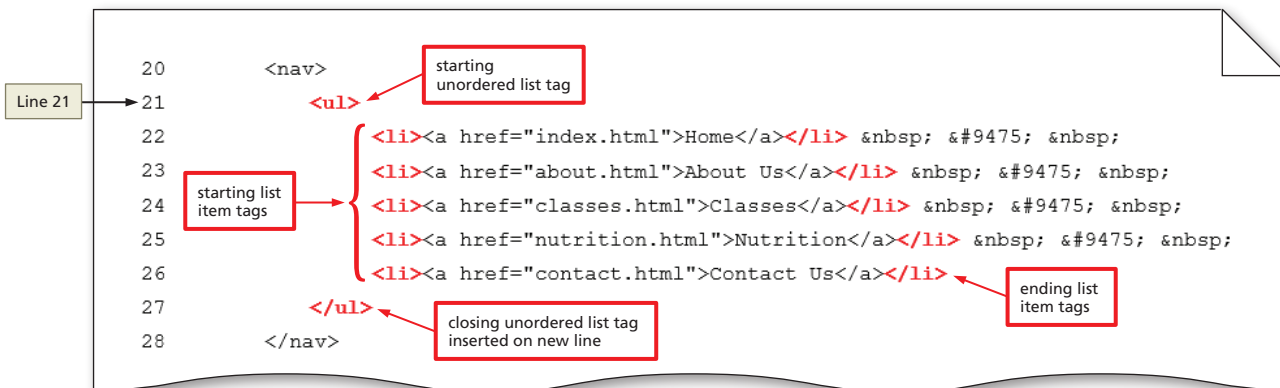


Figure 4–21

- 2
 - Delete the nonbreaking space characters, ` `, and vertical lines, `┃`, after each closing `</li>` tag (Figure 4–22).

Q&A Why am I deleting the nonbreaking space characters and vertical lines?
You will use CSS to style the padding and margin between these links so you no longer need the nonbreaking spaces or vertical lines to create space between the links.

**Figure 4-22****3**

- Copy the unordered list code in the <nav> section, starting with the opening tag on Line 21 and continuing through the ending tag on Line 27.
- Open the about.html file in your text editor.
- Select all elements within the <nav>...</nav> tags on Lines 21–25, and then paste the text from index.html containing the links coded as an unordered list (Figure 4-23).

**Figure 4-23****4**

- Open the contact.html file in your text editor.
- Select all elements within the <nav>...</nav> tags on Lines 21–25, and then paste the text from index.html containing the links coded as an unordered list (Figure 4-24).

**Figure 4-24**

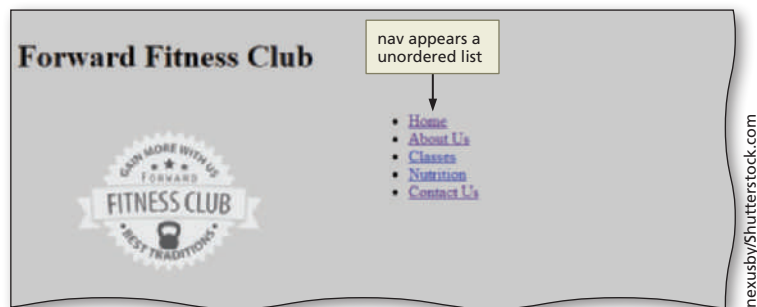
5

- Open the template.html template file in your text editor.
- Select all elements within the <nav>...</nav> tags on Lines 21–25, and then paste the text from index.html containing the links coded as an unordered list (Figure 4–25).

**Figure 4–25****6**

- Save and view all four HTML files in a browser to verify the list items appear as a bulleted list (Figure 4–26).

Q&A Why does each link appear on its own line in the nav area?
 Each link appears on its own line because the list item element is a block element.

**Figure 4–26**

CSS List Properties

To control the appearance of numbered and bulleted lists, you use the CSS **list-style properties**. By default, unordered lists display a solid bullet before each list item. Likewise, ordered lists display numbers (1, 2, 3, and so on) before each list item. You use the list-style-type property to specify a different type of bullet or a different numbering style to use in a list. For example, the following style rule defines a filled square bullet for an unordered list:

```
ul {
    list-style-type: square;
}
```

The following style rule defines uppercase Roman numerals for an ordered list:

```
ol {
    list-style-type: upper-roman;
}
```

If you want to display an image instead of a bullet, use the list-style-image property. To indicate the file name of the image file, begin the value with “url”

followed by the file name or path in parentheses. For example, the following style rule defines the image in the `arrow.png` file as the bullet for unordered lists:

```
ul {
    list-style-image: url(arrow.png);
}
```

To specify the position of the bullet or number in a list, use the `list-style-position` property. The default value for this property is **outside**, which displays the list item with a bullet or number outside of the list's content block as in the following text:

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
2. Morbi odio nisl, facilisis non egestas a, tristique vitae neque.

Using **inside** as the value displays the bullet or number inside the list's content block, as in the following text:

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
2. Morbi odio nisl, facilisis non egestas a, tristique vitae neque.

BTW

Lorem Ipsum

Web and print designers often use pseudo-Latin text that begins with "Lorem ipsum" as filler text when they want to focus on design or formatting rather than content.

BTW

List Properties

Appendix B provides a full description of the list properties and their valid values.

To Style the Navigation Using CSS

The default marker for each list item in an unordered list is a bullet. Add style rules to remove the bullet from the unordered list and to display the list items as an inline block so that they appear horizontally on the page. *Why?* For a desktop display, you want the links in the navigation bar to appear on a single line, and you do not want to display the bullet marker. Create three new style rules. First, create a style rule for the unordered list within the nav to remove the bullet and margin, and align text to the right. Next, create a style rule for list item elements within the nav to display them as an inline block and increase the font size. Finally, create a style rule for anchor elements within nav list item elements to display them as a block, change the text color to white, center-align the text, remove the underline, and specify padding. The following steps add three style rules to the style sheet to format the unordered list for the website.

1. Open the `styles.css` file, place the insertion point after the closing brace on Line 14, and then press the ENTER key twice to insert new Lines 15 and 16.
- On Line 16, type `nav ul {` to add the `ul` selector nested within a `nav` element and an opening brace.
- Press the ENTER key to add Line 17, press the TAB key to indent the line, and then type `list-style-type: none;` to add a declaration that removes the bullet marker.
- Press the ENTER key to add Line 18, and then type `margin: 0;` to remove margin.
- Press the ENTER key to add Line 19, and then type `text-align: right;` to align the unordered list to the right.
- Press the ENTER key to add Line 20, press the SHIFT+TAB keys to decrease the indent, and then type `}` to close the style rule (Figure 4-27).

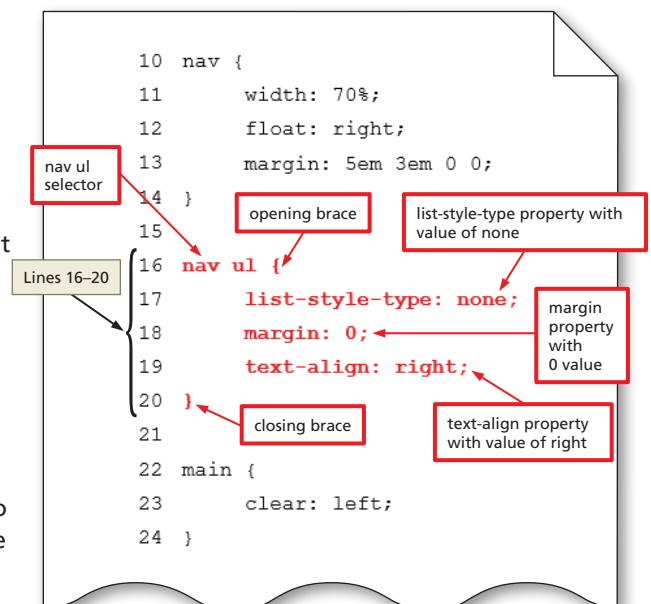


Figure 4-27

Q&A What is the `list-style-type` property?

The `list-style-type` property specifies the type of bullet for a list item. If you do not want to display a bullet, specify a value of `none` to remove the default bullet.

2

- Press the ENTER key twice to insert new Lines 21 and 22.
- Starting on Line 22, add the style rule for the nav li selector as shown in Figure 4–28 to style the list items within the nav element.

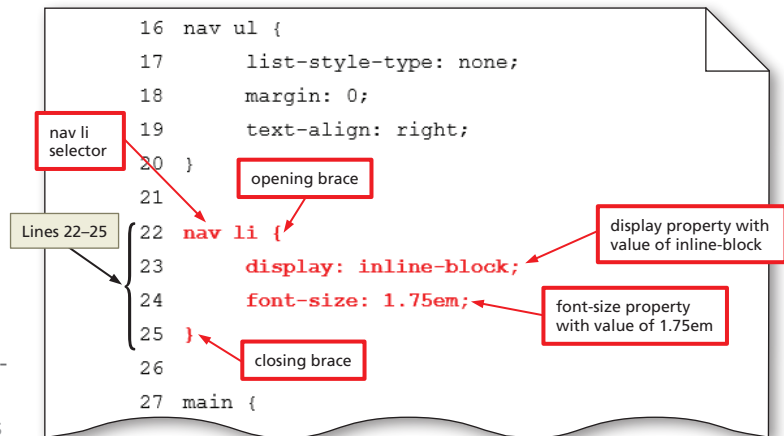
Q&A Why am I using the display property with a value of inline-block?

This will show the list items horizontally on the page instead of vertically.

What does the 1.75em value mean for the font-size property?

Setting the font-size property to 1.75em means the font size of the list item text is 1.75 times larger than the default font size of the browser.

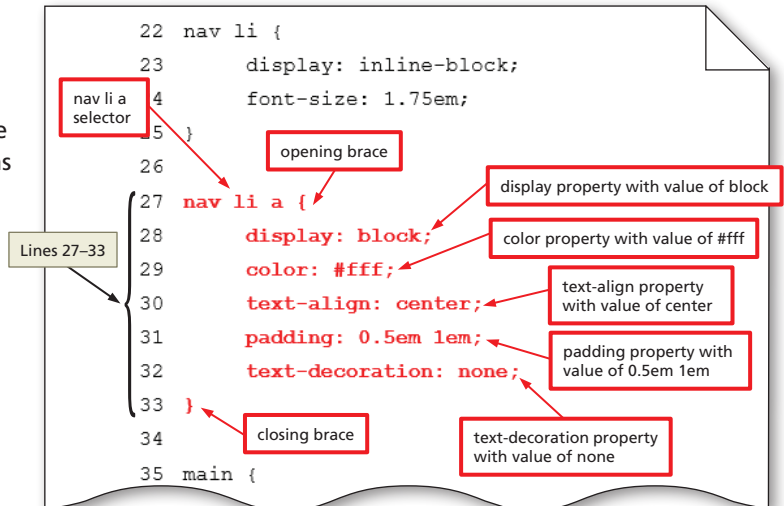
If the default size is 1.25em, for example, the font size for list items within the nav element will be 2.1875em.

**Figure 4–28****3**

- Press the ENTER key twice to insert new Lines 26 and 27.
- Starting on Line 27, add the style rule for the nav li a selector as shown in Figure 4–29 to style the anchor elements nested within the list items with the nav parent element.

Q&A What is the purpose of the text-decoration property?

The text-decoration property is used to remove the underline from the navigation links.

**Figure 4–29****4**

- Save the styles.css file and view the index.html page in your browser (Figure 4–30).

**Figure 4–30**

Making Responsive Images

Anyone viewing a webpage within a browser can adjust the width and height of the browser window. To ensure that images on your page adjust to the size of the browser window, you use responsive images in your design, which is major component of a responsive design strategy. You will learn more about responsive design in Chapter 5. A **responsive image** resizes itself to accommodate the size of the browser window. To create responsive images:

1. Delete the height and width attributes and values for the `img` tags in the HTML files.
2. Add a style rule for the `img` element in the CSS file to provide the desired flexibility such as the following style: **`max-width: 100%;`**

By setting the `max-width` of the image to 100%, the image automatically stretches to fill its parent element, up to 100% of its actual size. The height grows in proportion to the width. Use the `max-width` property instead of the `width` property to constrain the image to 100% of its *actual* size in case the browser window grows even larger. Stretching the image beyond its actual size degrades the quality of the image, even if the webpage is viewed on a very large widescreen monitor.

Figure 4–31 shows an example of a responsive image on the National Oceanic and Atmospheric Administration website. The browser window on the left is wider than the browser window on the right. The same image appears in both windows. Because it is a responsive image, it can grow to accommodate the larger browser window and shrink to accommodate the smaller browser window.

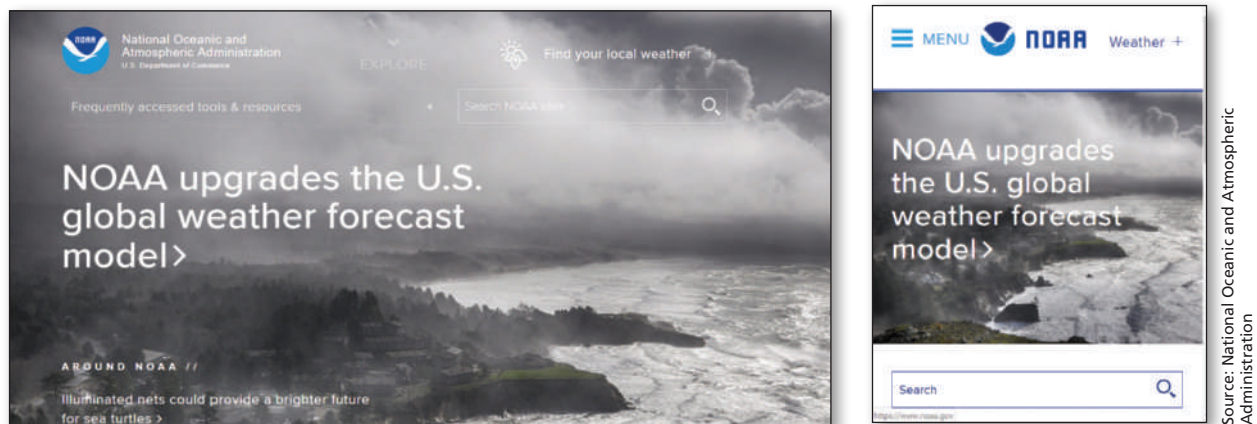


Figure 4–31

Recall that the wireframe shown in Figure 4–15 includes a hero image, which appears below the header and navigation elements. A **hero image** is a large banner image strategically placed on the website to capture the visitor's attention. Many of today's websites incorporate hero images within their design. Sometimes, the hero image is a large image displayed near the top of the webpage. Other times, the hero image consumes the entire background of the webpage. Figure 4–31 shows an example of a hero image.

According to the site design, you need to add a hero image to the Forward Fitness Club home page. You will add a `div` element to contain the image and then use CSS to make the image responsive.

To Add a Hero Image

Add a hero image to the home page of the Forward Fitness Club website. Before you can add the hero image, you need to acquire the hero-image.jpg file from the Data Files and copy it to your fitness/images folder. *Why? Capture the attention of your audience by including visual content to clarify the purpose of the website.* The following steps copy the hero-image.jpg file to the fitness/images folder and add the hero image to the home page of the Forward Fitness Club.

1

- Use File Explorer (Windows) or Finder (macOS) to navigate to the Data Files.
- Double-tap or double-click the chapter04 folder, double-tap or double-click the chapter folder, and then tap or click the hero-image.jpg file to select it.
- Copy the file to your fitness/images folder.
- If necessary, open the index.htm file in your text editor to prepare to insert a div element.
- Place your insertion point at the end of Line 28 and press the ENTER key twice to insert new Lines 29 and 30.
- On Line 30, type `<!-- Hero Image -->` to insert a comment.
- Press the ENTER key to insert a new Line 31, and then type `<div id="hero">` to add a starting div element with an attribute value of hero.
- Press the ENTER key to insert a new Line 32, press the TAB key to increase the indent, and then type `` to add an image element.
- Press the ENTER key to insert a new Line 33, press the SHIFT+TAB keys to decrease the indent, and then type `</div>` to close the div element (Figure 4–32).



Figure 4–32

2

- Delete the height and width attributes and values from the logo image on Line 16 (Figure 4–33).

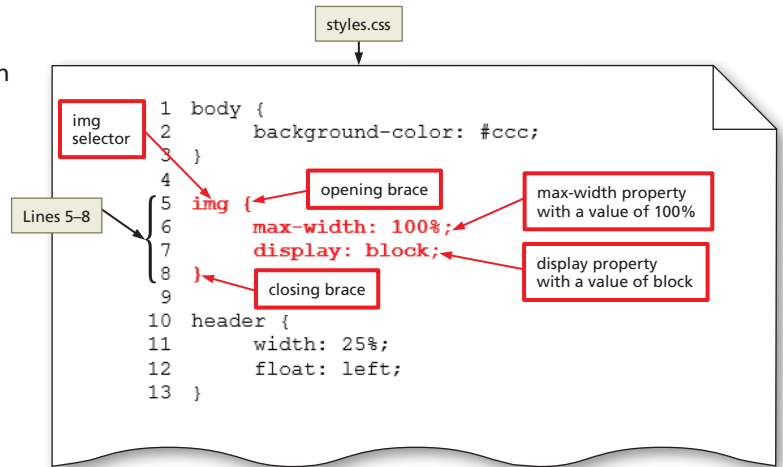
Q&A Should I delete the closing angle bracket for the img tag?
 No. Be careful to delete only the height and width attribute values and not the closing angle bracket (>) for the img tag.



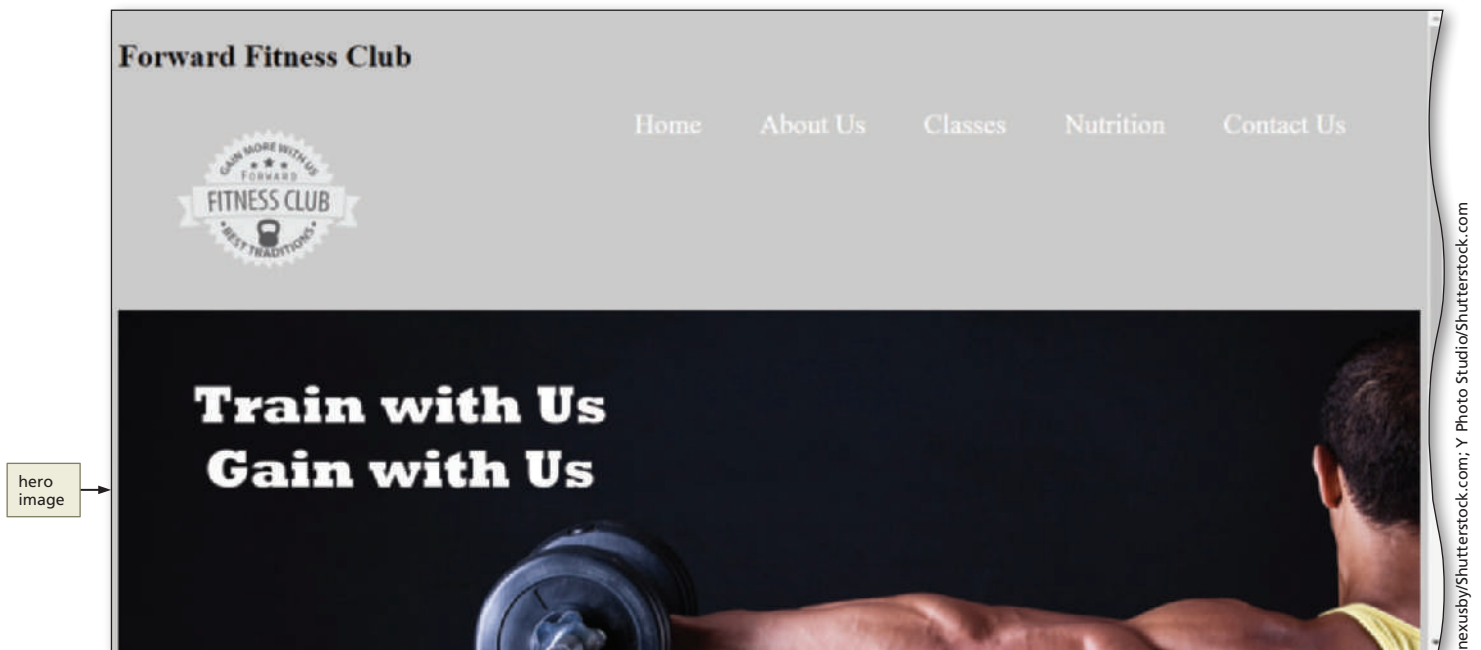
Figure 4–33

3

- Switch to the styles.css file, place your insertion point at the end of Line 3, and then press the ENTER key twice to insert new Lines 4 and 5.
- Starting on Line 5, add a style rule for the img selector as shown in Figure 4–34.

**Figure 4–34****4**

- Save changes to all files and then open index.html in your browser to view the page (Figure 4–35).

**Figure 4–35**

Q&A The hero image is so large that I must scroll down to see it. I also see the gray background color on either side of the image. Will I make more adjustments to the page to fix these flaws?

Yes. You are not done enhancing this webpage. You will make changes in future steps to improve the appearance of the home page.

5

- Resize the window to make it smaller to view how the image decreases in size (Figure 4–36).

Q&A How does the new style change the images on the webpage?

The logo image automatically resizes with the browser window, and the images on the index.html page also resize as the window becomes smaller than their natural size.

hero image decreases in size when the window is narrowed

The image is responsive, but now my navigation is no longer horizontal. How do I fix this?

Resize your window to the size of your desktop display. Do not be concerned about the navigation appearance when the window size is decreased. You will address this in Chapter 5.



Figure 4–36

To Remove Height and Width Attributes from img Elements

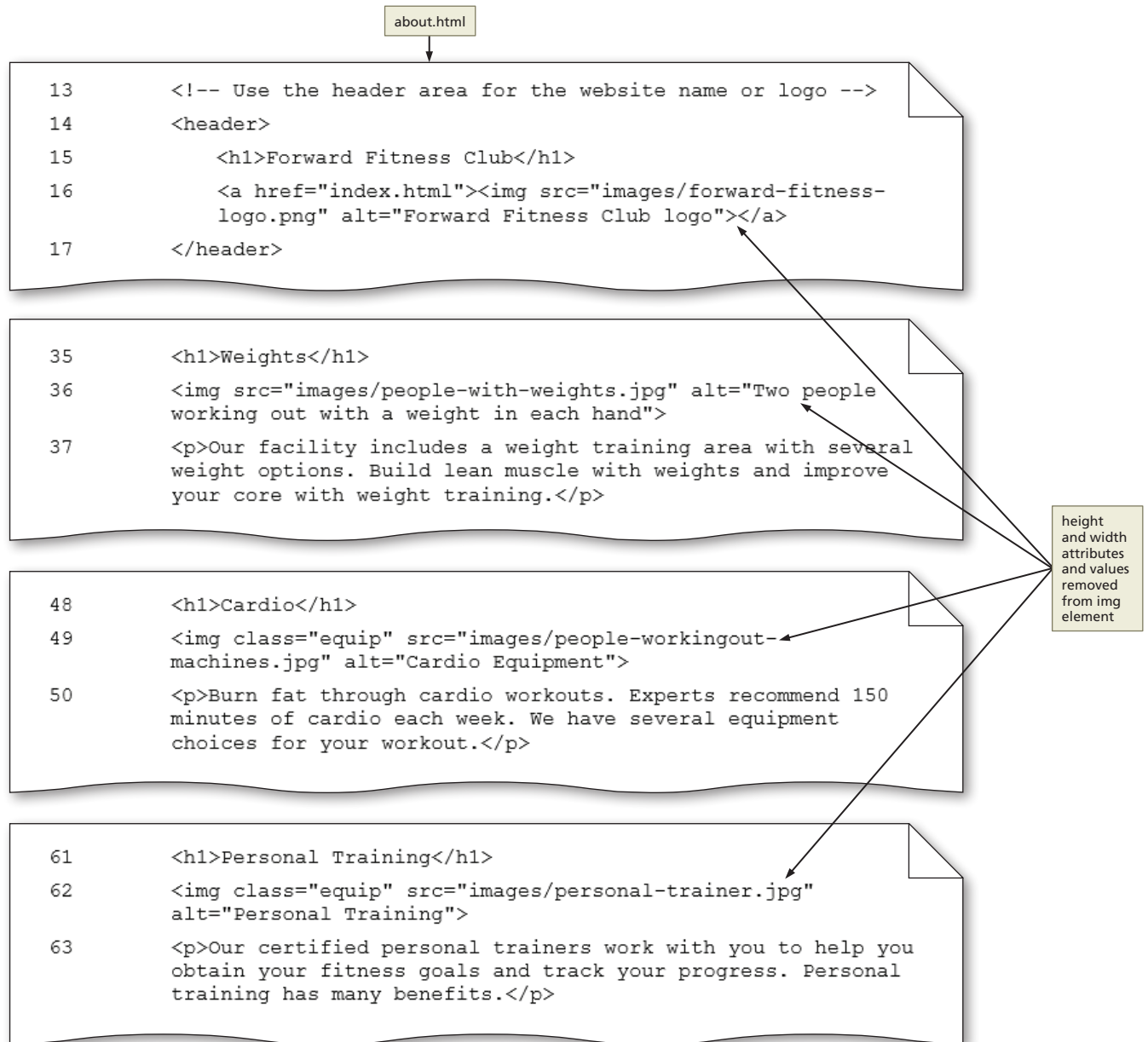
Remove the height and width attributes from image elements on the About Us, Contact Us, and template pages. *Why? You remove the height and width attributes and use CSS to create a responsive image that grows and shrinks with the size of the browser window.* The following steps delete the height and width attributes from the img tags in the HTML files.

1

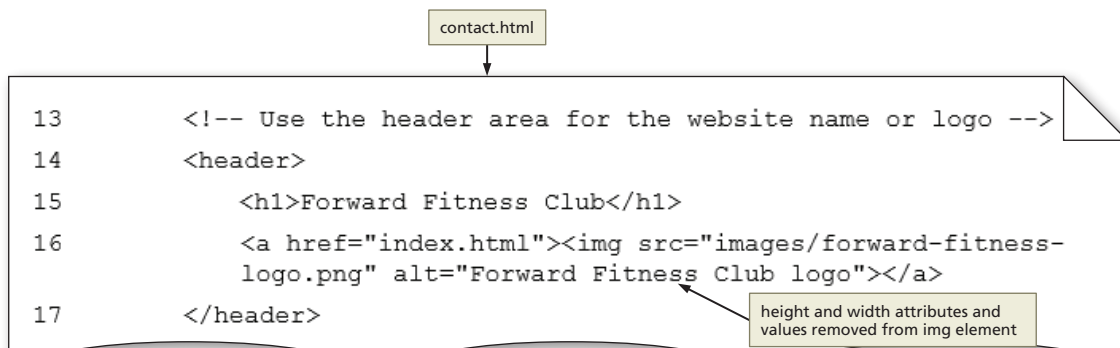
- Open the about.html, contact.html, and template.html files in your text editor.
- In the about.html file, delete the height and width attributes and values from the logo img element on Line 16.
- Delete the height and width attributes and values and from the other images on Lines 36, 49, and 62 (Figure 4–37).

Q&A Should I delete the alt attribute within the img tag?

No. Be careful to delete only the height and width attribute values and not the alt attribute for the img tag.

**Figure 4-37****2**

- Switch to the contact.html file.
- Delete the height and width attributes and values from the logo img element on Line 16 (Figure 4-38).

**Figure 4-38**

3

- Switch to the template.html file.
- Delete the height and width attributes and values from the logo img element on Line 16 (Figure 4–39).

template.html

```

13      <!-- Use the header area for the website name or logo -->
14      <header>
15          <h1>Forward Fitness Club</h1>
16          <a href="index.html"></a>
17      </header>
  
```

height and width attributes and values removed from img element

Figure 4–39

To Modify the Style Rule for the Main Element

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 **CREATE STYLE RULES**

4 ADD COMMENTS | 5 VALIDATE CSS FILE

You already have a style rule for the main element, but need to modify it to complete its design. *Why?* Currently, the style rule for the main selector only has one declaration. Add more declarations to enhance the appearance of the main content on the webpage. Next, add a declaration to specify the background color and padding, and increase the font size for the main element. You will also remove the clear property. The following steps modify the style rule for the **main** element.

1

- Switch to the styles.css file, place your insertion point at the end of Line 41, and then press the ENTER key to add Line 42.
- On Line 42, type **background-color: #fff;** to add a declaration that specifies the background color.
- Press the ENTER key to add Line 43, and then type **padding: 2%;** to add a declaration for the padding property.
- Press the ENTER key to add Line 44 and then type **font-size: 1.25em;** to add a declaration that specifies a font size (Figure 4–40).

styles.css

```

32  nav li a {
33      display: block;
34      color: #fff;
35      text-align: center;
36      padding: 0.5em 1em;
37      text-decoration: none;
38  }
39
40  main {
41      clear: left;
42      background-color: #fff;
43      padding: 2%;
44      font-size: 1.25em;
45  }
  
```

main selector

Lines 42–44

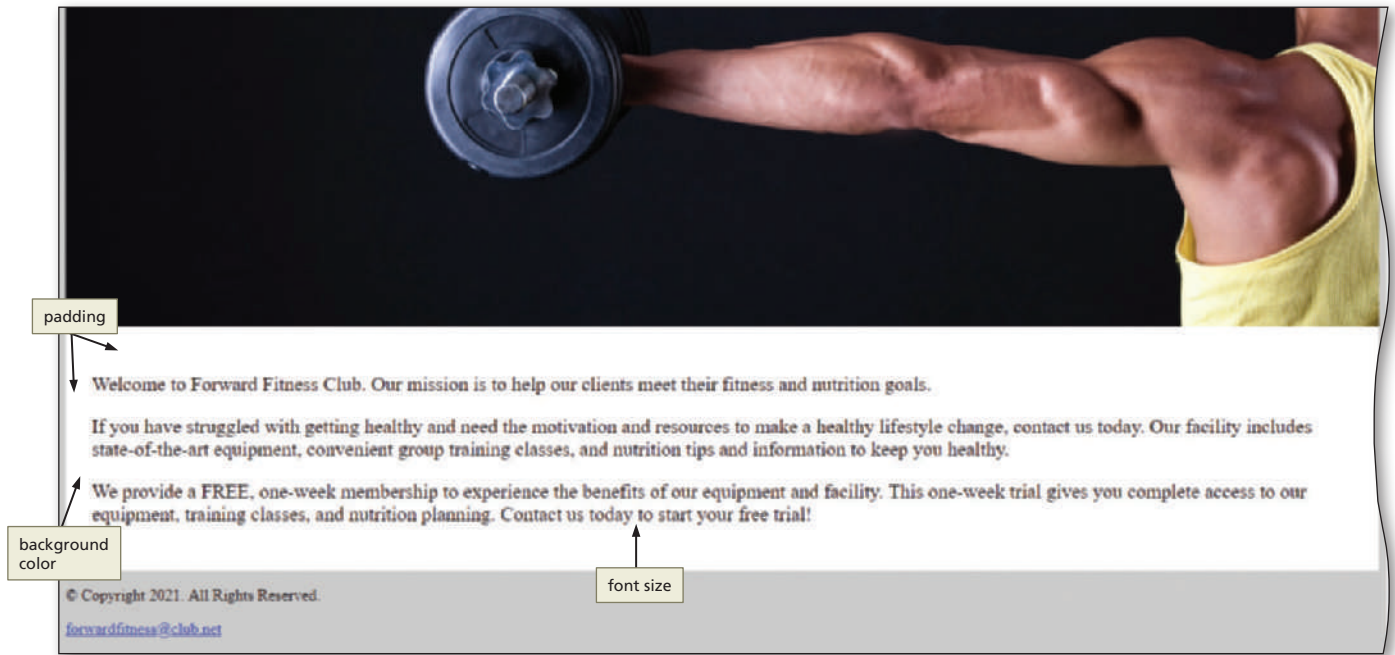
background-color property with a value of #fff

padding property with a value of 2%

font-size property with a value of 1.25em

Figure 4–40**2**

- Save your changes.
- View the home page in a browser to see the applied styles. If necessary, scroll down to view the content within the main element (Figure 4–41).

**Figure 4-41**

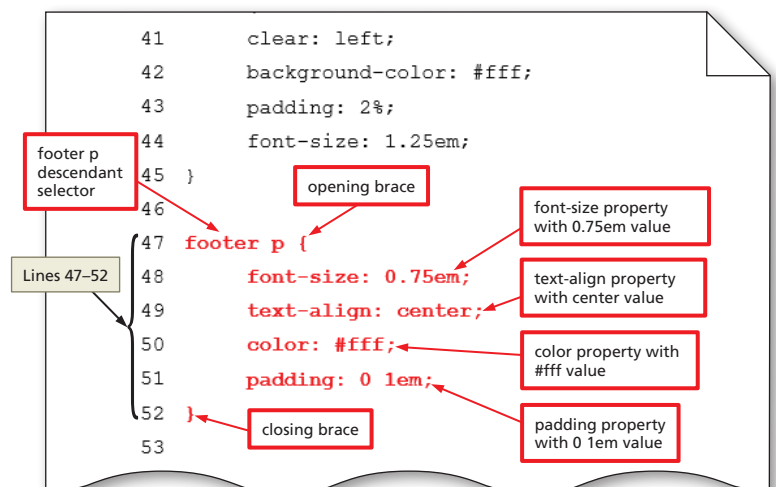
To Create a Style Rule for the Footer Element

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 CREATE STYLE RULES
4 ADD COMMENTS | 5 VALIDATE CSS FILE

The footer is located at the bottom of the webpage below the main area and contains paragraph elements with copyright information and an email address. Create two style rules to style the footer paragraph content and the email link. *Why?* The footer section provides supplemental information, so its text should be smaller than the other text on the page. It should also be centered and have some padding to improve its appearance. The following steps create styles for paragraph elements and links within the footer.

1

- In `styles.css`, place the insertion point after the closing brace on Line 45 and then press the ENTER key twice to insert new Lines 46 and 47.
- On Line 47, type `footer p {` to add the footer p selector and an opening brace.
- Press the ENTER key to add Line 48, press the TAB key to indent the line, and then type `font-size: 0.75em;` to add a declaration that sets the font size.
- Press the ENTER key to add Line 49 and then type `text-align: center;` to add a declaration for text alignment.
- Press the ENTER key to add Line 50 and then type `color: #fff;` to add a declaration that sets the color.

**Figure 4-42**

- Press the ENTER key to add Line 51 and then type `padding: 0 1em;` to add a declaration that sets the padding.
- Press the ENTER key to add Line 52, press the SHIFT+TAB keys to decrease the indent, and then type `}` to close the style rule (Figure 4-42).

2

- Press the ENTER key twice to insert new Lines 53 and 54.
- On Line 54, type `footer p a {` to add the footer p a selector and an opening brace.
- Press the ENTER key to add Line 55, press the TAB key to indent the line, and then type `color: #fff;` to add a declaration that sets the link color.
- Press the ENTER key to add Line 56, and then type `text-decoration: none;` to add a declaration that removes the underline from the email address.
- Press the ENTER key to add Line 57, press the SHIFT+TAB keys to decrease the indent, and then type `}` to close the style rule (Figure 4-43).

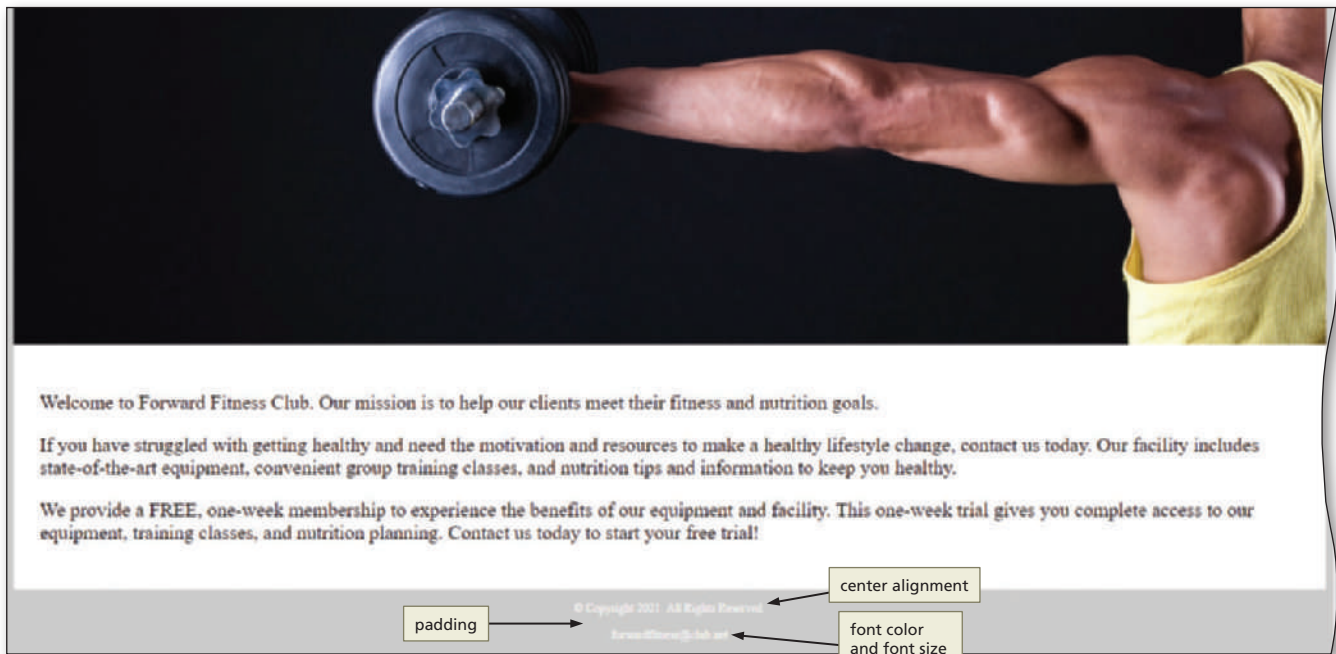
```

47 footer p {
48     font-size: 0.75em;
49     text-align: center;
50     color: #fff;
51     padding: 0 1em;
52 }
53
54 footer p a {
55     color: #fff;
56     text-decoration: none;
57 }

```

Figure 4-43**3**

- Save your changes.
- View the home page in a browser and, if necessary, scroll down to see the applied style (Figure 4-44).

**Figure 4-44**

Y Photo Studio/Shutterstock.com

Break Point: If you want to take a break, this is a good place to do so. You can exit the text editor now. To resume at a later time, run your text editor, open the file called `styles.css`, and continue following the steps from this location forward.

Creating Style Rules for ID Selectors

In Chapter 3, you created several id attribute values for div elements on every HTML page. Every page includes a primary div with an id attribute value of `container`, which holds all the webpage content. The About Us page includes several div elements with id attribute values. As discussed earlier in the chapter, you can style an id attribute by using an id selector for a style rule. You will style these id selectors to improve the layout of the content within the main element. The Contact page also includes div id attribute values. You need to create a style rule to format the div element nested within the main element on the Contact page. The following example creates a style rule for an id attribute value of `#contact`:

```
#contact {
    text-align: center;
}
```

The above style rule will center-align text within an element that has the id attribute value of `contact`.

To Create Style Rules for IDs on the About Us Page

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 CREATE STYLE RULES
4 ADD COMMENTS | 5 VALIDATE CSS FILE

The content on the About Us page is displayed in a single column. Improve the layout of this page by dividing the page content into three columns. *Why? Improving the layout of the webpage will improve the overall appearance and show more content within the same horizontal space.* Create four new style rules. First, create a style rule for the `weights`, `cardio`, and `training` id attributes to adjust their widths, apply a float, and set some margin. Next, create a style rule for the `exercises` id attribute to clear a float, set top and bottom borders, specify a background color, and set some padding. Create a style rule to format the description terms within the `exercises` id attribute to make the text bold. Finally, create a style rule for the description definitions within the `exercises` id attribute to specify some padding. The following steps add four style rules to the style sheet to format div id attributes, description terms, and description definitions on the About Us page.

- 1 In the `styles.css` file, place the insertion point after the closing brace on Line 45, and then press the ENTER key twice to insert new Lines 46 and 47.
- On Line 47, type `#weights, #cardio, #training {` to add id selectors for `weights`, `cardio`, and `training` and an opening brace.
- Press the ENTER key to add Line 48, press the TAB key to indent the line, and then type `width: 29%;` to add a declaration that specifies a width.
- Press the ENTER key to add Line 49, and then type `float: left;` to add a declaration that specifies a float.
- Press the ENTER key to add Line 50, and then type `margin: 0 2%;` to add a declaration that specifies margin.
- Press the ENTER key to add Line 51, press the SHIFT+TAB keys to decrease the indent, and then type `}` to close the style rule (Figure 4-45).

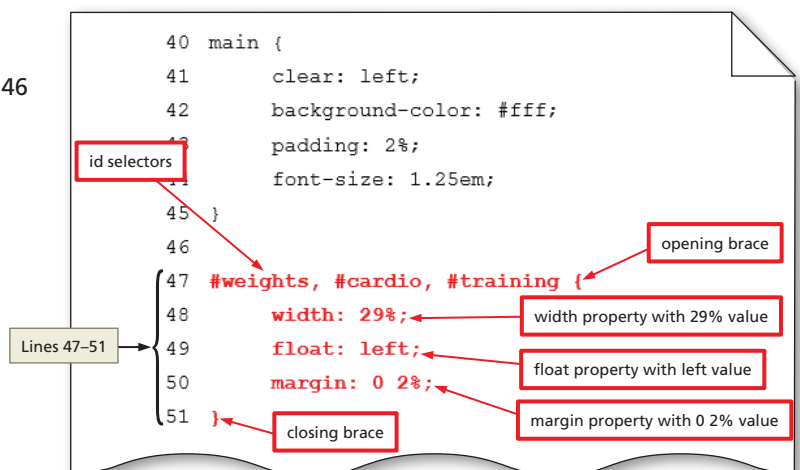


Figure 4-45

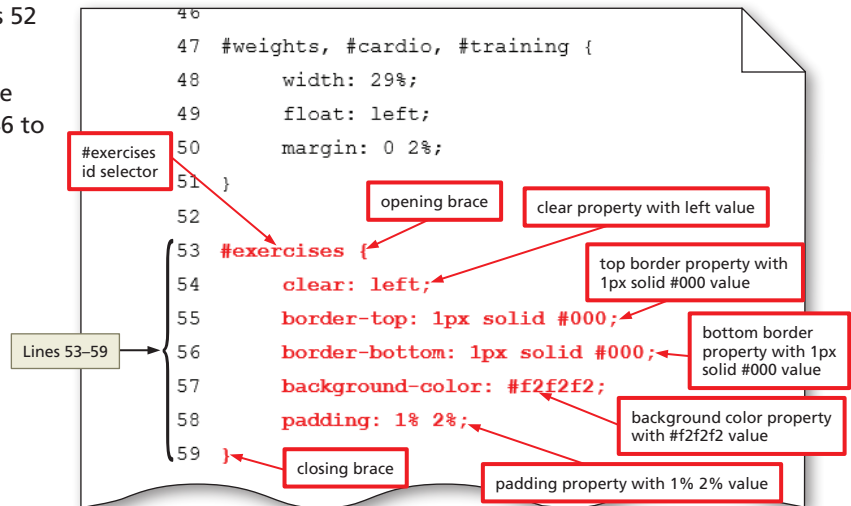
Q&A Why do I need to use the # symbol before weights, cardio, and training?
To style an id selector, you must begin with the # symbol.

How do you stop content from floating so that it starts on its own line?

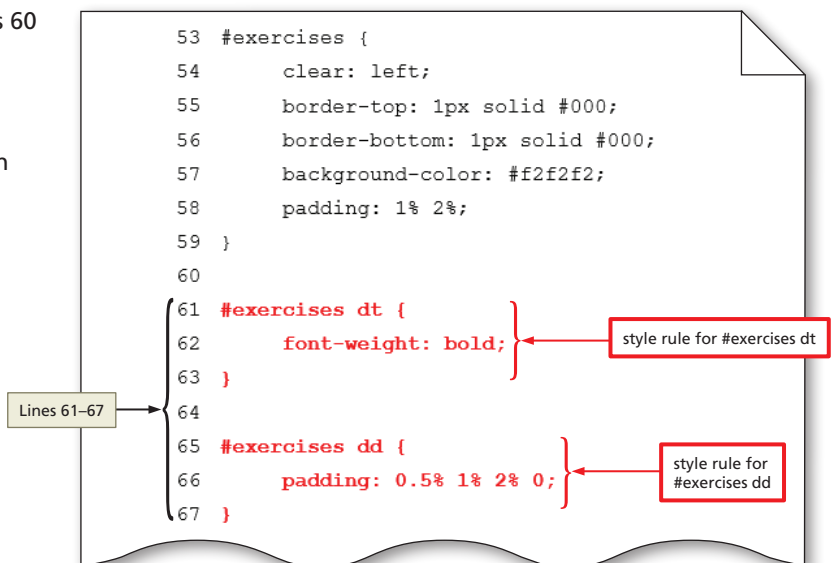
To stop floating an element, use the clear property. For example, apply the following declaration to the content that you want to start on its own line, or clear the float: clear: left; (clear left float), clear: right; (clear right float), or clear: both; (clear left and right float).

2

- Press the ENTER key twice to insert new Lines 52 and 53.
- Starting on Line 53, add the style rule for the #exercises id selector as shown in Figure 4-46 to style the exercises id attribute.

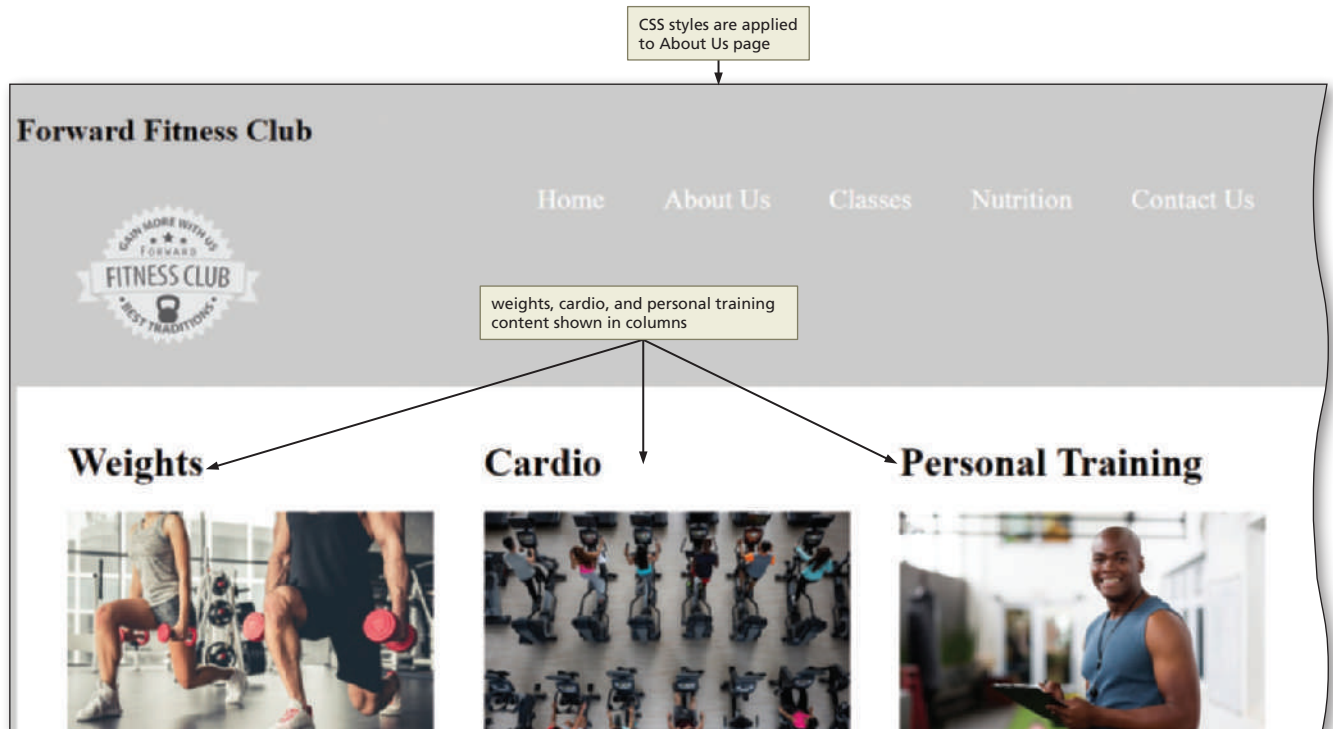
**Figure 4-46****3**

- Press the ENTER key twice to insert new Lines 60 and 61.
- Starting on Line 61, add the style rules for #exercises dt and #exercises dd selectors as shown in Figure 4-47 to style the description terms and description definitions within the exercises id attribute div element.

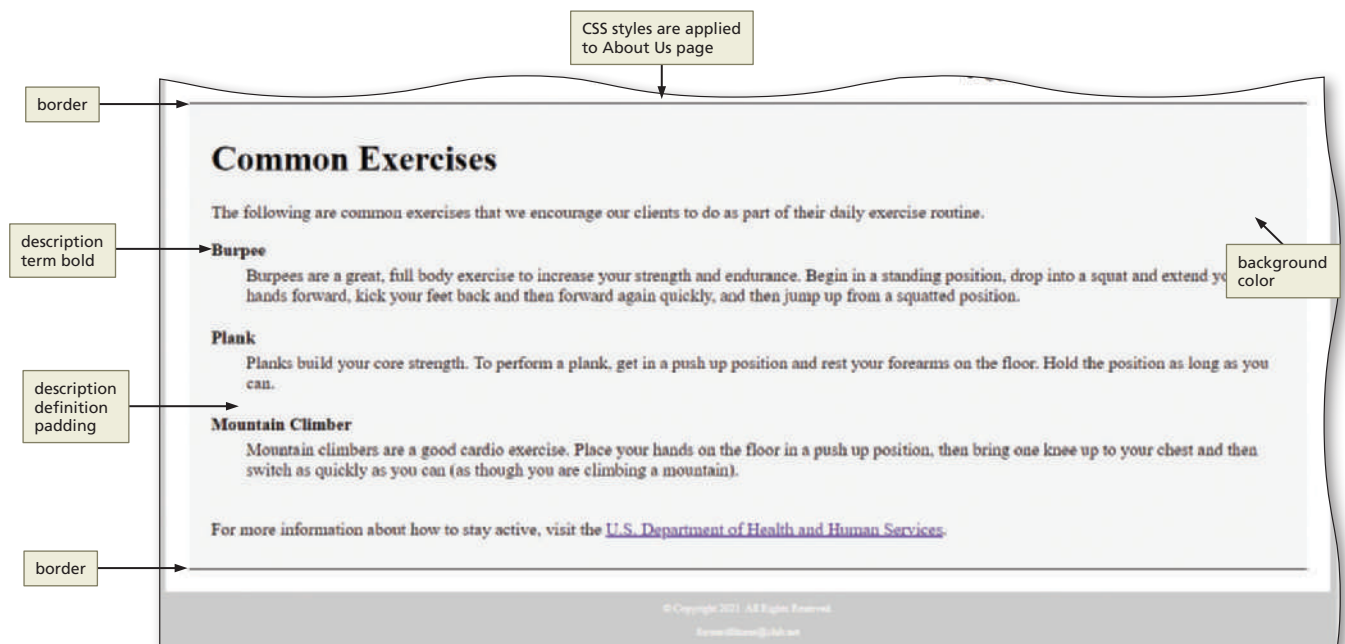
**Figure 4-47**

4

- Save the styles.css file and view the about.html page in your browser (Figure 4–48).

**Figure 4–48****5**

- Scroll down to view the changes made to the exercises id attribute, description terms, and description definitions (Figure 4–49).

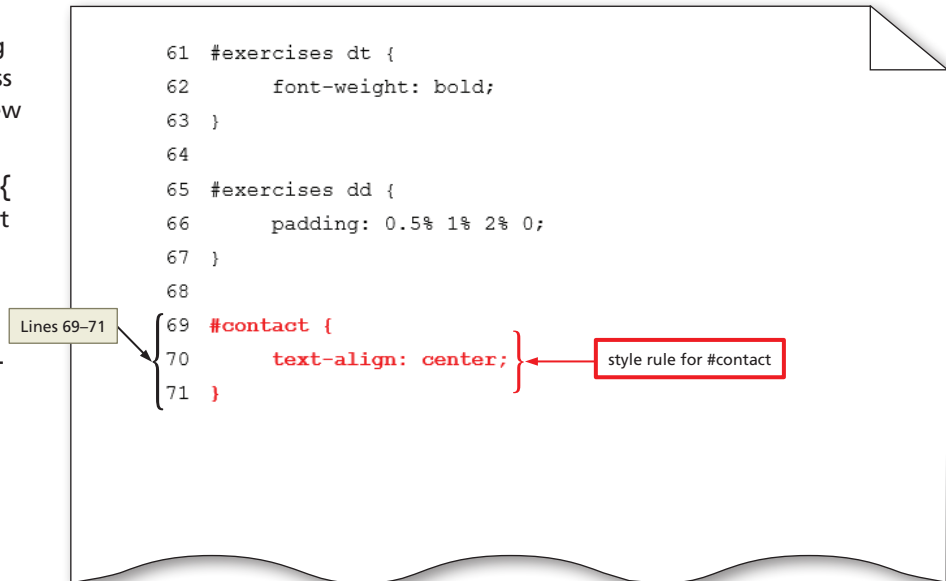
**Figure 4–49**

To Create Style Rules for IDs on the Contact Page

The content on the About Us page is aligned left. Change the text alignment to center the content within the main element. *Why? Centering the content will improve the overall page appearance.* Create two new style rules. First, create a style rule for the contact id attribute to center-align its content. Then, create a style rule for the email link within the contact id attribute to change its color and remove the underline. The following steps add two style rules to the style sheet to center-align content and format the email link within the main content area on the Contact page.

1

- In the styles.css file, place the insertion point after the closing brace on Line 67, and then press the ENTER key twice to insert new Lines 68 and 69.
- On Line 69, type `#contact {` to add an id selector for contact and an opening brace.
- Press the ENTER key to add Line 70, press the TAB key to indent the line, and then type `text-align: center;` to add a declaration that center-aligns content.
- Press the ENTER key to add Line 71, press the SHIFT+TAB keys to decrease the indent, and then type `}` to close the style rule (Figure 4–50).



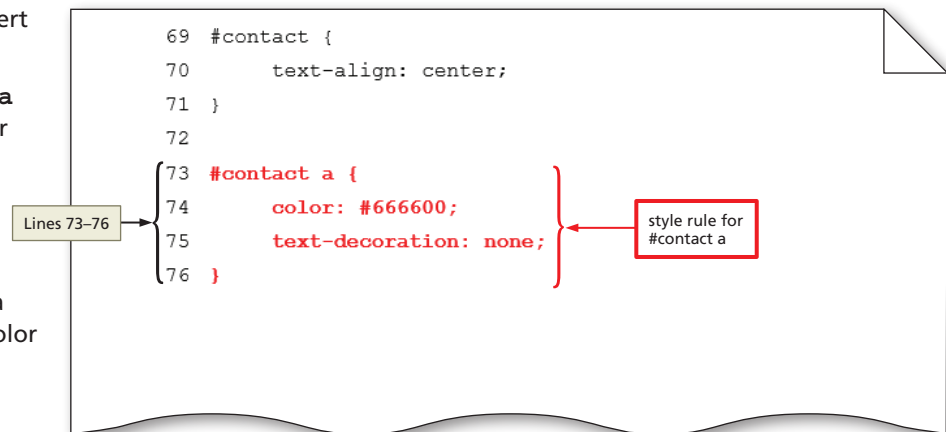
```

61 #exercises dt {
62     font-weight: bold;
63 }
64
65 #exercises dd {
66     padding: 0.5% 1% 2% 0;
67 }
68
69 #contact {
70     text-align: center;
71 }
  
```

Figure 4–50

2

- Press the ENTER key twice to insert new Lines 72 and 73.
- On Line 73, type `#contact a {` to add a descendant selector and an opening brace.
- Press the ENTER key to add Line 74, press the TAB key to indent the line, and then type `color: #666600;` to add a declaration that specifies the color of the text.
- Press the ENTER key to add Line 75, and then type `text-decoration: none;` to add a declaration that removes the underline.
- Press the ENTER key to add Line 76, press the SHIFT+TAB keys to decrease the indent, and then type `}` to close the style rule (Figure 4–51).



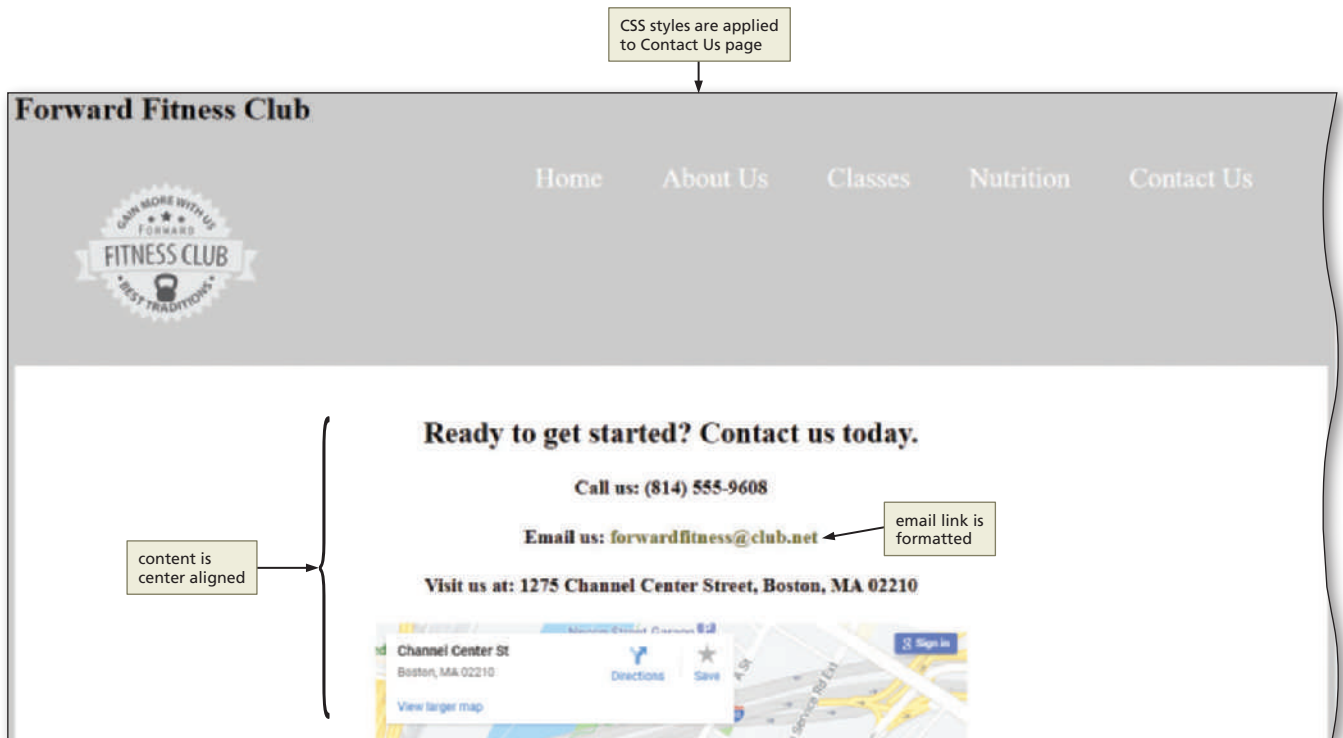
```

69 #contact {
70     text-align: center;
71 }
72
73 #contact a {
74     color: #666600;
75     text-decoration: none;
76 }
  
```

Figure 4–51

3

- Save the styles.css file and view the contact.html page in your browser (Figure 4–52).

**Figure 4–52**

Creating Class Attributes

Another commonly used attribute in HTML is the **class** attribute. Unlike the **id** attribute, a class attribute name can be applied to more than one **div** or other HTML element on a webpage. Classes provide another level of control over the styling or formatting of specific elements on a webpage. For example, rather than having all paragraphs of text appear with the same formatting, you might want to format call-to-action words different from the other paragraphs of text. To accomplish this, you can wrap the call-to-action words within a **span** element that uses a class attribute. You use a **span element** to group inline elements together on a webpage. You can then apply specific styling or formatting for the class. Using classes allows you to selectively apply style rules to specific HTML content. For example, if you want to draw the user's attention to specific page content, such as "start your free trial!", you can wrap this text within a **span** element, include a class attribute, and then create a class selector to style the call-to-action content. The following example shows text wrapped within a **span** element that includes the class attribute name **action**:

```
<span class="action">start your free trial!</span>
```

Because the above **span** element includes a class attribute, you can create a style rule to style the content within the **span**. For example, the following style rule uses a

class selector, `.action`, that sets the font size to 1.35em, the font color to #666600, and a font weight of bold:

```
.action {
    font-size: 1.35em;
    color: #666600;
    font-weight: bold;
}
```

To indicate you are using a class name as a selector, include a period (.) before the class name, similar to the way you include a number sign (#) with an id selector.

Any word can be used as a class name, as long as it does not contain spaces. Use a descriptive class name. When a class name has more than one word, include hyphens between each word and use all lowercase text. For example, if you want to apply a class to elements related to workout equipment, use the class name, `workout-equip`. Using names that describe the purpose makes the code easier to read and more flexible.

To Create and Style the action Class

1 CREATE CSS FILE | 2 LINK PAGES TO CSS FILE | 3 CREATE STYLE RULES

4 ADD COMMENTS | 5 VALIDATE CSS FILE

The home page includes the text, “start your free trial!” Enhance this as a call-to-action item by wrapping it in a span element and then assigning a class attribute to the span element to style the text. *Why? Styling the call-to-action text differently from the rest of the paragraph will attract the audience’s attention.* The following steps create a span element and a style rule for a class selector.

1

- Switch to `index.html` and place your insertion point before the word “start” on Line 44 to prepare to insert a span element.
- Type `<span class="action">` to insert an opening span tag with a class attribute value of `action`.
- Place your insertion point to the right of the exclamation point after the word `trial`, and then type `</span>` to insert a closing span tag (Figure 4–53).

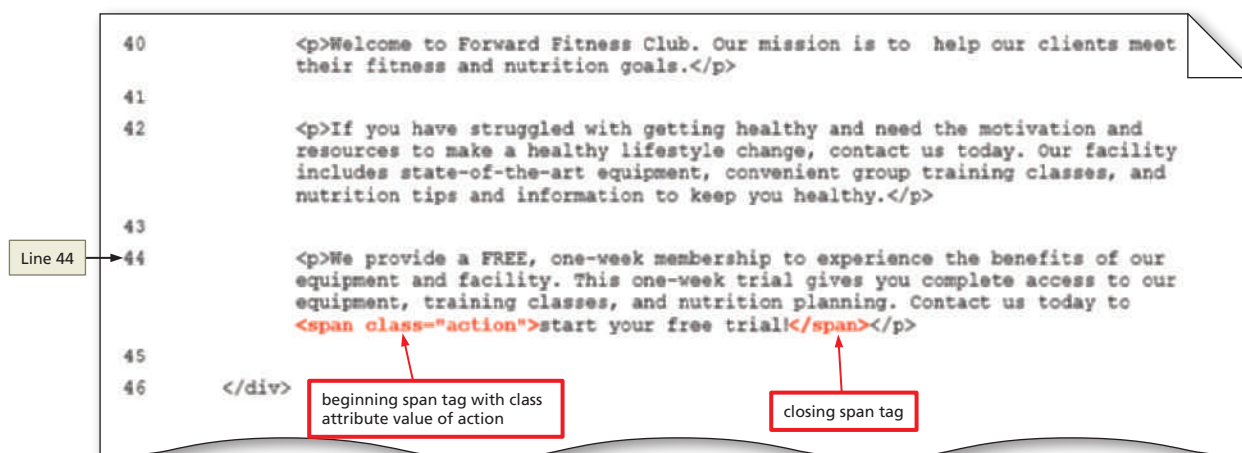
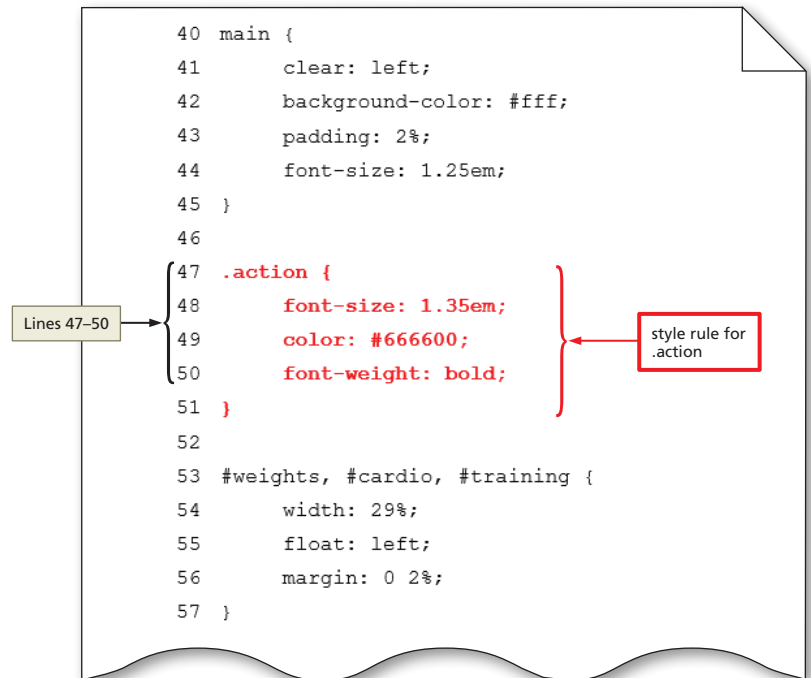


Figure 4–53

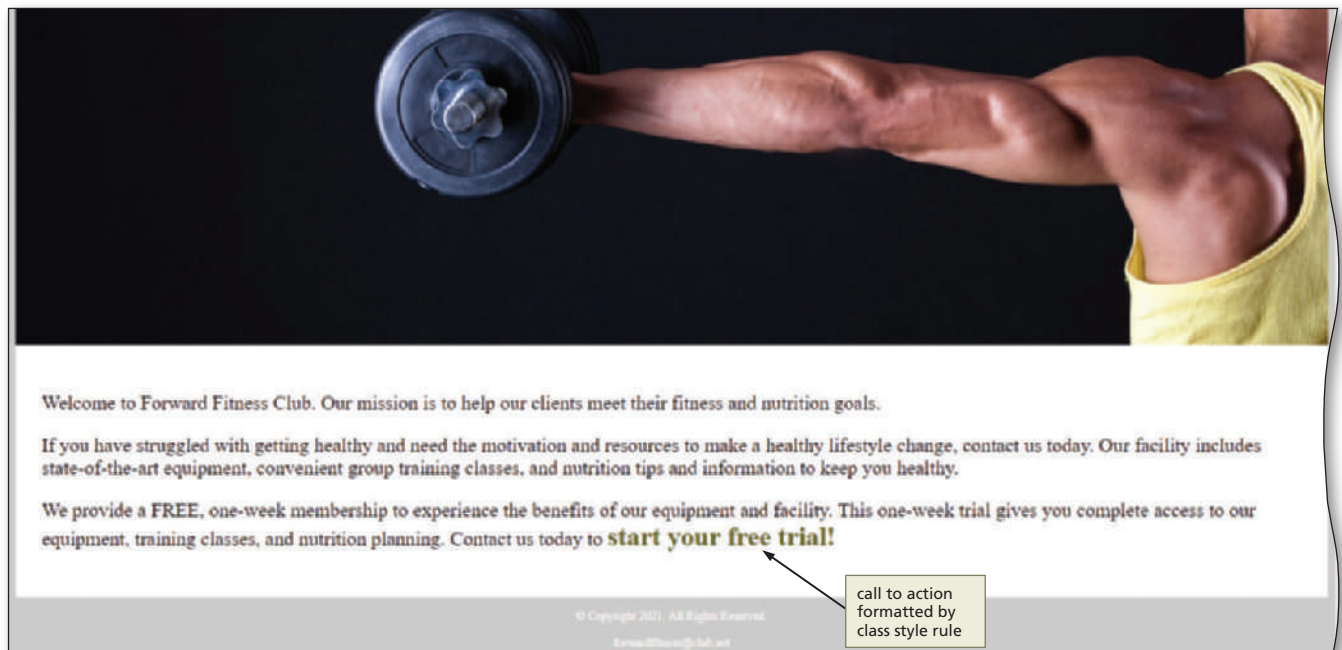
2

- Switch to `styles.css`, place your insertion point after the closing brace on Line 45, press the ENTER key twice to insert new Lines 46 and 47, and then type `.action {` to add a class selector and opening brace.
- Press the ENTER key to add Line 48, press the TAB key to increase the indent, and then type `font-size: 1.35em;` to add a declaration for font size.

- Press the ENTER key to add Line 49 and then type `color: #666600;` to add a declaration for the font color.
- Press the ENTER key to add Line 50 and then type `font-weight: bold;` to add a declaration for the font weight.
- Press the ENTER key to add Line 51, press the SHIFT+TAB keys to decrease the indent, and then type `}` to close the style rule (Figure 4–54).

**Figure 4–54****3**

- Save all changes and view index.html in your browser. If necessary, scroll down to view the call-to-action text (Figure 4–55).

**Figure 4–55****Q&A** Why does the .action selector start with a period?

To style a class, you insert a period before the class name to specify the selector in the CSS file.

Why am I adding the style rule for the class selector after the main style rule?

It is a common practice to keep style rules grouped together based on the HTML page layout. Because the action class is within the main element on the home page, this style rule is placed after the main style rule.